

Métodos exatos

Relatório completo

Makespan e custo de energia em máquinas paralelas não relacionadas

TÓPICOS ESPECIAIS EM SISTEMAS INTELIGENTES:
OTIMIZAÇÃO MULTI-OBJETIVO

Backtracking | CP-SAT | Programação dinâmica própria

Análise do modelo, conformidade, métricas das 17 entradas, fronteiras, experimentos prolongados e catálogo de 111 funções em 17 arquivos.

Os algoritmos próprios usam somente a biblioteca padrão. Matplotlib e ReportLab são ferramentas de apresentação. A documentação distingue prova de otimalidade, candidato viável e execução em andamento.

Gerado em 01/10/2026 00:08 (America/Sao_Paulo).

Snapshot dos dados: 2026-10-01T00:08:30.738951-03:00

Sumário

1. Conclusão e alcance	3
2. Problema, convenções e conformidade com o artigo	4
3. Por que os três algoritmos são corretos	5
4. Quantas possibilidades existem e quanto as regras reduzem	6
5. Resultados, certificações e eficiência observada	8
6. O que o backtracking efetivamente reduziu	10
7. Validação e auditorias independentes	11
8. Reprodutibilidade, limpeza e próximos usos	12
9. Avaliações independentes e correções	13
10. Fronteiras encontradas	14
11. Makespan mínimo por DP própria	17
12. Fronteira com código próprio	19
13. Backtracking e podas seguras	23
14. Leitura e desenho das fronteiras	27
15. MILP anterior: referência de biblioteca	29
16. Testes do backtracking	32
17. Testes de makespan e do MILP anterior	33
18. Testes da solução inteiramente própria	35
19. Gerador desta documentação	36
20. Modelo compartilhado e persistência	38
21. Método de biblioteca: CP-SAT	42
22. Contagem do espaço de busca	45
23. Validação, comparação e relatório	47
24. Controle das rodadas prolongadas	50
25. Testes dos contadores combinatórios	52
26. Teste de persistência no Windows	53
27. Testes da certificação CP-SAT	54

1. Conclusão e alcance

As três soluções são razoáveis e exatas para o modelo compartilhado: backtracking próprio com propagação, CP-SAT do OR-Tools e DP própria com epsilon-restrição. A DP de makespan é a primeira etapa da terceira solução. Os métodos próprios usam somente biblioteca padrão. O MILP anterior é complementar e não participa da comparação principal. Exatidão do algoritmo não implica conclusão dentro de qualquer orçamento: o backtracking permanece exponencial e só certifica a fronteira quando termina; as DPs limitam tarefas para evitar execução impraticável.

A eficiência foi demonstrada nas entradas pequenas do set1. CP-SAT e backtracking materializam até $O(n*m*o*H)$ opções individuais; o circuito do CP-SAT usa $O(m*n^2)$ arcos, e o fechamento de separações do backtracking custa $O(m*n^3)$. Nas entradas grandes, memória e preparação também limitam esses códigos, além da busca combinatória. Não apresentamos as três implementações atuais como soluções práticas para 750 tarefas; as grandes entradas receberam métricas analíticas, não fronteiras exatas inventadas.

Foram removidos apenas artefatos vazios verificados e temporários de apresentação. Entradas, resultados, baselines, testes e checkpoints são necessários às análises e retomadas. Os arquivos complementares MILP e suas dependências têm finalidade histórica explícita.

2. Problema, convenções e conformidade com o artigo

Cada tarefa escolhe exatamente uma máquina, modo e início inteiro. Duração = $\text{ceil}(\text{processing}/\text{speed})$. Tarefas disponíveis em zero, sem preempção e sem setup inicial. Máquinas não relacionadas, setups dependentes da máquina e dos jobs consecutivos. O setup independe do modo. Processamento em $[\text{start}, \text{end})$; ponta informada inclusiva é convertida para $[\text{peak_start}, \text{peak_end} + 1)$. $H = n_day * (hl + 1)$, portanto $H = 1440$ em set1 e vários dias em set2. Energia somente no processamento; setup e espera não cobram energia. O custo é avaliado com frações/inteiros escalados. max_cost normaliza TEC, sem restringir viabilidade.

Fontes locais: artigo-ideia-mhm-upms.pdf, pp.4-7 e 9-10; Atividade_01___MHM.pdf, pp.2-6. Esses documentos fundamentam o modelo; suas descrições não substituem o pedido do usuário. A imagem só nomeia as entregas e o relatório inicial não contém rubrica completa: as notas dos agentes avaliam a conformidade verificável, sem certificação de critérios pedagógicos não fornecidos.

A Eq.(4) do artigo aplica restrições entre pares e exige desigualdade triangular envolvendo setup e processamento (p.7). Nosso modelo aplica setup entre consecutivos diretamente. Condição suficiente verificada: $S_i(a,b) \leq S_i(a,v) + \min_l d_i(v,l) + S_i(v,b)$, em trios distintos. Isso assegura que a cadeia de jobs intermediários também satisfaz a separação par-a-par. Nenhuma violação ocorreu nas cinco entradas set1. No set2 a condição não foi verificada; a correção operacional por adjacência independe dela. Não afirmamos reprodução literal daquela formulação nas entradas grandes.

Tarefas	Trios/máquinas testados	Violações
6	240	0
7	420	0
8	672	0
9	1008	0
10	1440	0

O artigo admite a discretização em minutos no modelo, mas seus experimentos exatos usam $\text{sizeD} = 144$, intervalos de dez minutos, onze pesos $\alpha = 0, 0.1, \dots, 1$ e limite $800 * n * \ln(m)$ segundos por peso (p.10). As execuções publicadas usaram C++/Gurobi7.0.2 e i7-4510U/16GB/Ubuntu19.10 (p.13). Usamos Python, minuto, epsilon-restrição e outro hardware. Comparações diretas de tempos, cardinalidade ou HV não medem uma reprodução equivalente. Soma ponderada pode perder pontos não suportados, portanto ter mais pontos não prova superioridade isoladamente. As tabelas/figuras do artigo são referência metodológica; reproduzi-las numericamente exige converter dados, grade, unidades, normalização e protocolo, com novos hashes.

3. Por que os três algoritmos são corretos

Backtracking: uma opção por tarefa já impõe as regras one-hot, e o domínio só inclui termos dentro de H . A propagação exclui sobreposição e separação mínima impossível. O fechamento de caminhos usa processamento mínimo dos intermediários e é otimista, inclusive para setups não triangulares. Uma lacuna só é excluída por setup irreparável se nenhum job futuro couber nela. Limites de carga e custo ignoram restrições adicionais e são inferiores seguros. Dominância usa apenas soluções viáveis encontradas pela própria busca, sem sementes dos outros métodos. Checkpoints preservam a pilha exata.

CP-SAT: intervalos opcionais, AddExactlyOne e AddNoOverlap modelam alocação e processamento. Um circuito por máquina representa consecutivos e permite máquina vazia. AddElement avalia o custo inteiro por início. A cada rodada minimiza makespan, depois energia com makespan $\leq$ ótimo encontrado; ambos precisam de OPTIMAL para arquivar o ponto. Decrementar a energia escalada em uma unidade exige melhora estrita sem erro de float. INFEASIBLE no próximo teto prova ausência de solução mais barata; não invalida as soluções anteriores. FEASIBLE e UNKNOWN deixam apenas candidato sem prova. O limite total, workers, seed, conflitos e branches agora são registrados; orçamento é cooperativo e pode ser ultrapassado por preparação/operação indivisível.

DP própria: para makespan isolado, modo mais rápido e ausência de espera são suficientes. A recorrência por máquina/subconjunto/último encontra a melhor sequência e a partição minimiza o máximo. Para energia, guarda todos os rótulos não dominados (end, TEC) de cada estado com último job. Um rótulo anterior e mais barato pode esperar e imitar qualquer continuação, por isso domina com segurança. Para o mesmo modo, um início posterior de custo igual ou maior também é dominado; next_cheaper salta apenas esses casos. Últimos jobs só se fundem quando a máquina terminou. A partição considera todos os subconjuntos disjuntos para cada epsilon. Limites: 16 tarefas makespan, 12 energia, com reconstrução das tabelas ao retomar.

Por que $C_{max} \leq \epsilon$: epsilon é orçamento de prazo. Se a solução ótima de energia termina antes, ela continua admissível e não precisa de espera artificial. Minimizar energia para todos os tetos inteiros $C_{min}..H$ e manter os pares não dominados recupera toda fronteira discreta. Pontos repetidos são normais. A varredura DP faz todos os epsilons; CP-SAT usa epsilon no custo e salta entre pontos via otimizações lexicográficas.

4. Quantas possibilidades existem e quanto as regras reduzem

$B = n * m * H$ o células binárias: uma matriz irrestrita tem 2^B possibilidades. Isso inclui muitas matrizes sem sentido, como nenhum ou vários inícios para a mesma tarefa. Ao impor exatamente uma escolha por tarefa, o espaço torna-se $A = (m * H)^n$. Ao exigir término válido individual, $D_j = \sum_i (i, l) \max(0, H - d_{jil} + 1)$; $D = \prod_j D_j$. Nenhum dos três métodos enumera as 2^B matrizes. Células de início inválido equivalem a bits fixados em zero, não a jobs removidos.

n	B	A (one-hot)	D (individual)	Redução $A \rightarrow D$
6	51840	4.15989e+23	3.41058e+23	18.012652%
7	60480	3.59414e+27	2.78368e+27	22.549458%
8	69120	3.10534e+31	2.37053e+31	23.662802%
9	77760	2.68301e+35	1.93713e+35	27.800159%
10	86400	2.31812e+39	1.60468e+39	30.776579%

O total F de escalonamentos que respeitam TODAS as regras é contado exatamente no set1, antes de qualquer dominância. DP de contagem $C_i[S, \text{last}, \text{end}]$: para anexar job j/modo l com término e, somar $C_i[S, \text{last}, t]$ para $t \leq e - d_{jil} - S_i(\text{last}, j)$. Prefixos calculam essa soma. O estado vazio tem término0 e contagem1, sem setup inicial. Somar últimos/términos produz $A_i(S)$; somar produtos $A_i(S_i)$ sobre todas as partições disjuntas produz F. Modos distintos contam como opções distintas, mesmo com mesmo tempo/custo. Uma atribuição tem ordem única na máquina porque durações são positivas e não há sobreposição. Nenhum rótulo é descartado por dominância nessa contagem.

Verificação combinatória: para sequência/modos fixos, k jobs e carga L (processamentos+setups), a máquina admite $\text{binom}(H-L+k, k)$ vetores de inícios, se $L \leq H$. Esses vetores distribuem a espera antes e entre jobs; cauda livre fecha a contagem. Não há prova de ótimo por conhecer F: o melhor par ainda depende da energia.

n	Sem sobreposição (exato)	Com setups F (exato)	Redução $D \rightarrow F$
6	2.04352e+23	1.93838e+23	43.165560%
7	1.23514e+27	1.13661e+27	59.168594%
8	8.70586e+30	7.81995e+30	67.011830%
9	4.80383e+34	4.16595e+34	78.494247%
10	2.69980e+38	2.25761e+38	85.931116%

Inteiros completos, não arredondados, estão em search_space.json e search_space.csv. As razões de redução usam D como denominador na segunda tabela e A na primeira. Os testes conferem a contagem com enumeração nas seis fixtures, incluindo setups reparáveis e dois dias.

No set2, a contagem exata exponencial não foi executada. Calculamos A e D exatamente e um teto U para F usando pares disjuntos (0,1),(2,3),... sem sobreposição: escolhas em máquinas diferentes são compatíveis; na mesma máquina contamos analiticamente as duas ordens. Para inícios x em [0,A-1], y em [0,B-1], $y \geq x+g$: $t = \min(A, \max(0, B-g))$; $\text{count} = t * (B-g) - t * (t-1)/2$. Multiplicar contagens de pares disjuntos (e o job restante, se houver) dá U. Ignorar setups e interações entre pares só amplia o conjunto, logo $F \leq U \leq D$.

A redução reportada é pelo menos $1-U/D$, não uma estimativa de F . O teto depende do pareamento fixo declarado; não é necessariamente o melhor limite possível.

Entrada	H	D	Teto U	Redução mínima $D \rightarrow U$
50_10_1439_5_S_1-124	1440	1.32969e+242	1.12052e+242	15.730856%
50_10_1439_5_S_1-9	1440	1.32962e+242	1.12049e+242	15.728763%
50_20_1439_5_S_1-124	1440	1.44073e+257	1.32025e+257	8.362035%
50_20_1439_5_S_1-9	1440	1.44073e+257	1.32025e+257	8.362035%
250_10_1439_5_S_1-9	1440	3.08241e+1210	1.27144e+1210	58.751667%
250_10_1439_5_S_1-124	4320	2.22858e+1332	1.66186e+1332	25.429250%
250_20_1439_5_S_1-9	1440	6.03873e+1285	3.89751e+1285	35.458118%
250_20_1439_5_S_1-124	2880	9.12605e+1362	7.33616e+1362	19.612907%
750_10_1439_5_S_1-9	2880	1.22907e+3863	3.29767e+3862	73.169360%
750_10_1439_5_S_1-124	18720	3.89680e+4477	3.18623e+4477	18.234816%
750_20_1439_5_S_1-9	2880	6.82852e+4088	3.52842e+4088	48.328228%
750_20_1439_5_S_1-124	8640	3.11347e+4450	2.49959e+4450	19.717019%

O expoente de 2^B é preservado sem materializar esse inteiro enorme. Alguns produtos D têm mais de 4300 dígitos; a exportação local permite esses inteiros. Casas científicas das tabelas servem para apresentação; CSV/JSON preservam o valor exato. Os horizontes multi-dia são calculados dos dados, sem inferir H pelo 1439 no nome.

5. Resultados, certificações e eficiência observada

Entrada	Método	Status	Pontos	Tempo s	ϵ da DP	HV/HV completo
6	cp_sat_epsilon_constraint	complete	43	1141.529	0	1.000000
10	cp_sat_epsilon_constraint	incomplete	6	630.546	0	0.834634
8	cp_sat_epsilon_constraint	incomplete	16	630.344	0	0.916513
9	cp_sat_epsilon_constraint	incomplete	14	630.735	0	0.894408
6	custom_exact_pagination	incomplete	14	4095.250	0	0.791837
7	custom_exact_pagination	incomplete	17	3780.281	0	0.890563
10	epsilon_exact_dp	complete	97	103.391	1271	1.000000
7	epsilon_exact_dp	complete	41	40.875	1317	1.000000
8	epsilon_exact_dp	complete	57	54.844	1310	1.000000
9	epsilon_exact_dp	complete	86	62.453	1290	1.000000
7	cp_sat_epsilon_constraint	complete	41	1260.203	0	1.000000
6	epsilon_exact_dp	complete	43	22.265	1347	1.000000

A coluna epsilon da DP conta apenas os tetos de makespan resolvidos nessa varredura. Zero no CP-SAT não indica falta de prova: ele percorre limites de custo e certifica os pontos por outra enumeração.

Tempos são registrados localmente, com execuções concorrentes; não publicamos speedup controlado. Baselines históricos podem ter outro contexto. A execução CP-SAT histórica de 6 levou 1141,529s (~19,0min), 87 chamadas e 43 pontos completos. O backtracking histórico com apenas poda de sobreposição levou 1111,176s (~18,5min), examinou 223.388.972 folhas candidatas e encontrou zero escalonamentos válidos devido aos setups; terminou pelo orçamento, não provou inviabilidade. A DP própria histórica de 6 concluiu 43 pontos em 22,265s. São observações, não um confronto de desempenho com protocolo igual.

As contagens e estados atuais foram verificadas racionalmente. O custo foi reavaliado a partir de potência, modo, tarifa e duração, junto com unicidade dos jobs, intervalos, horizonte e setups. Fronteiras completas da mesma entrada devem coincidir exatamente; pontos provados parciais devem pertencer à referência completa quando ela existe. Epsilons da DP devem ser contínuos e sua energia deve coincidir com o ótimo da referência. Baselines sem escalonamento são referências de pares, não recebem validação de schedule inexistente.

Tarefas	Cmax mínimo	Maior Cmax Pareto	Menor TEC	Pontos completos
6	94	220	134.099428000	43
7	124	329	146.992858800	41
8	131	215	190.709143200	57

Tarefas	Cmax mínimo	Maior Cmax Pareto	Menor TEC	Pontos completos
9	151	352	194.327955400	86
10	170	415	185.976602000	97

Limitação experimental adicional: nas cinco entradas set1, a primeira ponta começa no intervalo 1080 e todos os pontos da fronteira completa terminam antes dele. Assim, a troca entre makespan e TEC observada nesses casos decorre de máquinas, modos e durações, sob tarifa fora de ponta; esses dados não evidenciam um ganho por adiar processamento para outra tarifa. A implementação de tarifa variável é exercitada pelas fixtures e testes adversariais, inclusive ponta antecipada e preços invertidos. Os dados grandes com múltiplos dias exigiriam outro orçamento/escala de resolução para avaliar esse efeito em experimentos oficiais. Isso não invalida as fronteiras pequenas nem autoriza atribuir seus ganhos à tarifa de ponta.

HV usa o mesmo ponto estrutural por entrada: $R_C = H + 1$; $R_E = 1 + \sum_j \max_{(i,l)} [d_{jl} \pi_i \lambda_l \max_{tarifa} 24 / \text{slots_per_day}]$. Isso é pior que toda solução viável e independe do método. O HV é calculado racionalmente em 2D; HV/HV_completo mede área relativa sob esse RP fixo, não uma probabilidade de sucesso. O RP do artigo para 6, (250, 239.91), não domina nosso ponto (94, 242.2037755), portanto não reutilizamos seus HV sem adequar protocolo e referência.

6. O que o backtracking efetivamente reduziu

n	Tempo s	Decisões	Folhas	Pontos provisórios	Cobertura D
6	4095.2	1066682	69	14	0.002446192600%
7	3780.3	3617571	118	17	0.002602440925%

Para cada ramo eliminado, o contador usa o produto das quantidades originais dos jobs restantes. Isso representa subárvores brutas disjuntas da árvore adaptativa, incluindo opções que outras regras também rejeitariam. A atribuição do motivo é a primeira poda aplicada, não uma decomposição causal única. Identidade validada: cobertura=folhas+propagação+setup irreparável+dominância. O trabalho real é o número de decisões e folhas efetivamente examinadas. Dominância pode eliminar soluções viáveis que não acrescentam um ponto melhor; não é uma regra de inviabilidade. Toda poda depende do prefixo e da fronteira encontrada; a ordem pode alterar a eficiência.

n	Combinações cobertas por regras	Combinações cobertas por dominância	Ainda não cobertas	Pontos dominados por referência
6	909775111847624596	7433180618770943553	341050481749058534399942	14
7	7421133951002765764387	65022718222023310867111	2783616188019182065310697664	17

Mesmo podando bilhões de combinações, o restante do produto continua gigantesco. Backtracking evita matrizes binárias inválidas desde a representação, mas uma árvore de inícios por minuto contém muitas permutações, atribuições e esperas. Limites inferiores separados de custo/makespan podem ser muito otimistas e o percurso inicial pode encontrar soluções longe dos extremos. Cobertura não indica percentual do Pareto, não permite extrapolar linearmente o tempo restante e não certifica completude. Não chamamos seus pontos provisórios de pontos Pareto globais.

Foram iniciadas duas retomadas independentes, 6 e 7 tarefas, com 10800 segundos adicionais cada. Partiram dos próprios checkpoints, com 375s e 60s acumulados. Relatórios/checkpoints são copiados para a nova rodada e gravados a cada minuto; manifestos amostram evolução. Se o processo ainda estiver ativo no snapshot, as três horas são orçamento solicitado, não tempo já realizado. Manifestos possuem PIDs, horários UTC e status. Não se deve iniciar nova rodada sobre os mesmos arquivos enquanto esses processos estiverem ativos. Os dados de convergence.csv permitem avaliar HV e cardinalidade ao longo do tempo, com referência fixa.

7. Validação e auditorias independentes

A primeira auditoria matemática executou 13 testes do projeto com todas as bibliotecas presentes e 80 novos casos adversariais (seeds 1701..1780), 3 jobs, 1..3 máquinas, 2 modos e H entre 3 e 8. Sessenta casos tinham setups não triangulares e 24 eram inviáveis. Um oráculo independente enumerou 971.013 atribuições e calculou energia por slot sem o gerador compartilhado: as três soluções coincidiram; a DP foi verificada em cada epsilon. Após incluir métricas, recuperação de gravação e projeção do CP-SAT, 19 testes passaram integralmente. As quatro primeiras DPs pararam por `PermissionError` transitório do Windows durante os `.replace`; foram retomadas do último epsilon salvo após adicionar `retry` limitado, sem apagar o JSON anterior. Manifestos mantêm tentativas falhas e retomadas. Novas verificações e notas finais ficam em `docs/avaliacoes_agentes.json`.

Segunda rodada: conformidade 9,5/10, algoritmos 9,7/10 e experimentos 9,4/10. Os três agentes superaram o limiar solicitado. A auditoria matemática confirmou 160 contagens em 80 casos por sequências/modos e binomial, 80 tetos de pares por enumeração e os inteiros de ausência de sobreposição e viabilidade da entrada real de seis tarefas por procedimento independente. A auditoria experimental verificou 12 resultados e 413 schedules em seu snapshot, além de 58 amostras de HV não decrescente. Resultados posteriores ao snapshot desses agentes são validados pelo mesmo script; não são atribuídos retroativamente às auditorias.

Primeira rodada: conformidade 8,4/10, algoritmos 9,5/10, experimentos 8,2/10. Correções: README/fluxo próprio, limites total/seed/branches CP-SAT, motivo de parada na retomada, contagem viável exata e limites grandes, CSVs com denominadores, referências HV válidas, metadados/convergência, novas entradas e validação racional. Essas alterações preservam as podas já corretas; modificar um algoritmo apenas para obter nota maior não seria justificável sem uma falha matemática ou gargalo demonstrado.

8. Reprodutibilidade, limpeza e próximos usos

Máquina: AMD Ryzen5 3500X,6cores/6threads, Windows10.0.26200; Python3.11.9, OR-Tools9.15.6755, Matplotlib3.10.3. SciPy1.16.0 é dependência apenas do experimento MILP antigo. ReportLab é usado no PDF. Manifestos preservam versões e comandos; results_summary.json preserva hashes dos códigos, e cada resultado possui hash do input. Horários fonte sãoUTC; o snapshot do relatório usaAmerica/Sao_Paulo.

Artefatos de análise: search_space.json/csv, results_summary.json/results.csv, convergence.csv, plots por entrada, manifestos, logs e checkpoints. O PDF agrega análise e catálogo de funções. Os arquivos temporários de renderização não constituem evidência experimental e podem ser removidos depois da inspeção visual. A rubrica completa da disciplina e uma reprodução convertida para a grade do artigo permanecem limites do alcance, não evidências de correção pendentes.

Comandos: python src/analysis_metrics.py --resume; python src/analyze_results.py; python -m unittest discover -s tests -v; gerarPDF com docs/build_function_guide.py usando runtime com ReportLab. Para continuar um solver incompleto já parado, usar--resume no diretório da rodada; guardar novo orçamento e workers. Não reexecutar launch/run_experiments no mesmo manifesto.

9. Avaliações independentes e correções

article_conformity - conformidade científica: 8.4/10 para 9.5/10

As três soluções são razoáveis. As diferenças do artigo, a grade temporal, a interpretação de setups, as hipóteses de energia e a classificação das provas foram documentadas. Nenhuma correção matemática obrigatória restante.

Zero violações da condição triangular em todos os trios distintos das cinco entradas set1. No set2 a equivalência literal com a equação entre pares não foi verificada.

Artigo usa intervalos de dez minutos, onze pesos de soma ponderada e outro protocolo/hardware. A análise deixa explícito que a comparação atual é metodológica.

Fronteiras completas observadas: 43, 41, 57, 86 e 97 pontos. A documentação apresenta backtracking e DP próprios e CP-SAT como método de biblioteca.

Limitação declarada: a imagem e o relatório inicial não fornecem a rubrica completa da disciplina.

algorithms_audit - algoritmos, contagens e persistência: 9.5/10 para 9.7/10

Podas, DPs, CP-SAT e certificação são corretos para o modelo compartilhado. Contagens e limites são seguros. A gravação preserva a versão anterior; a projeção do primeiro CP-SAT sem energia mantém o makespan ótimo.

Primeira rodada: 80 instâncias adversariais, 971.013 atribuições independentes, 60 casos não triangulares e 24 inviáveis; todas as fronteiras e cada epsilon da DP coincidiram.

Segunda rodada: 160 contagens em 80 casos, verificadas por sequências/modos e $\text{binom}(H-L+k,k)$; 80 tetos por pares disjuntos conferidos por enumeração direta.

Instância real de seis tarefas: contagens 204352907588226627038820 sem sobreposição e 193838873473549160572392 com setups confirmadas por procedimento combinatório independente em 41,937 s.

Os 17 números de dígitos binários coincidiram com Decimal de precisão 50. Cinco testes CP-SAT atualizados passaram, incluindo projeção, candidato FEASIBLE com custo real e motivo de retomada.

Retry Windows: até 12 tentativas, espera máxima aproximada 3,62 s, sem apagar o destino anterior. Um escritor por arquivo continua sendo requisito do protocolo.

experimental_audit - experimentos e métricas: 8.2/10 para 9.4/10

As métricas agora distinguem os espaços, os denominadores, as reduções por regras e dominância, as provas e os dados concorrentes. Snapshots e referência de HV fixa resolvem as pendências de comparação. Nenhuma falha matemática restante.

17 entradas: $F \leq \text{sem-sobreposição} \leq U \leq D \leq A$ em set1, e $U \leq D \leq A$ em set2.

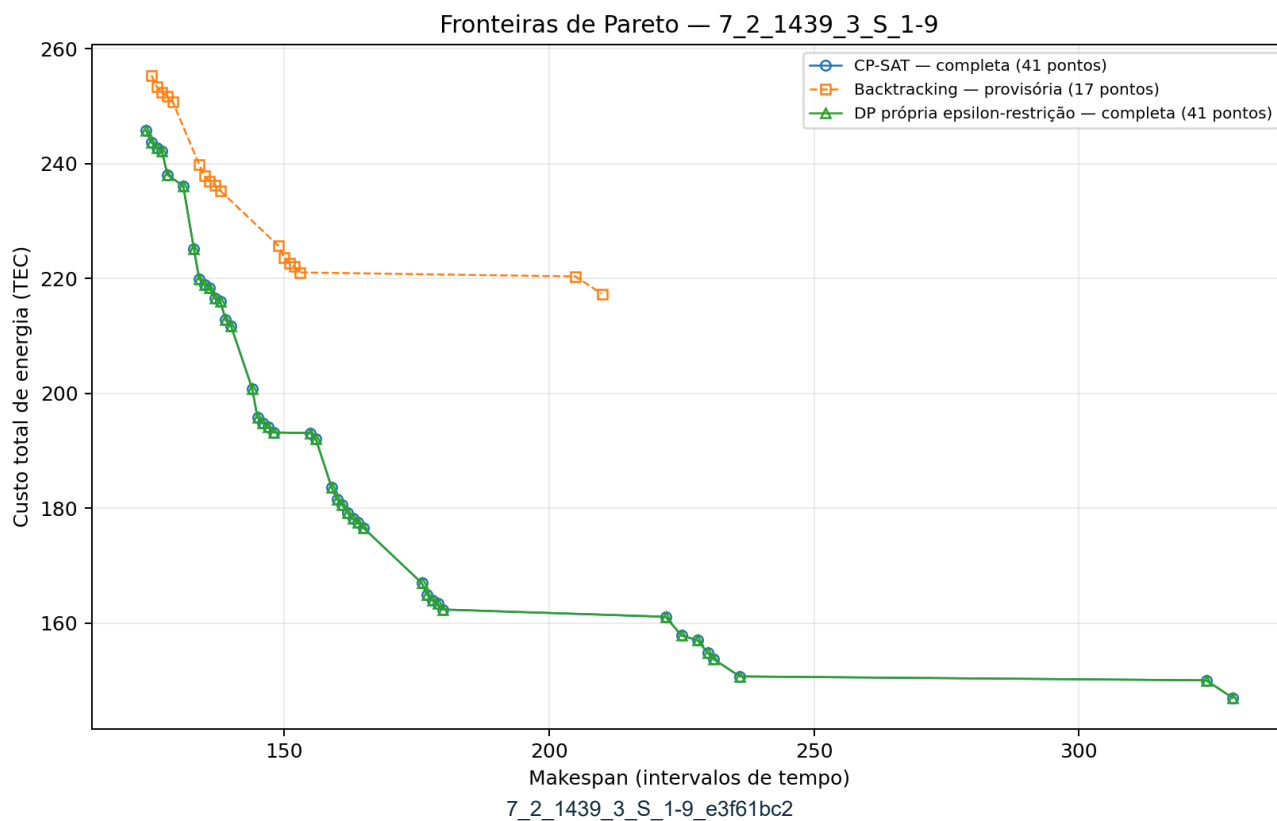
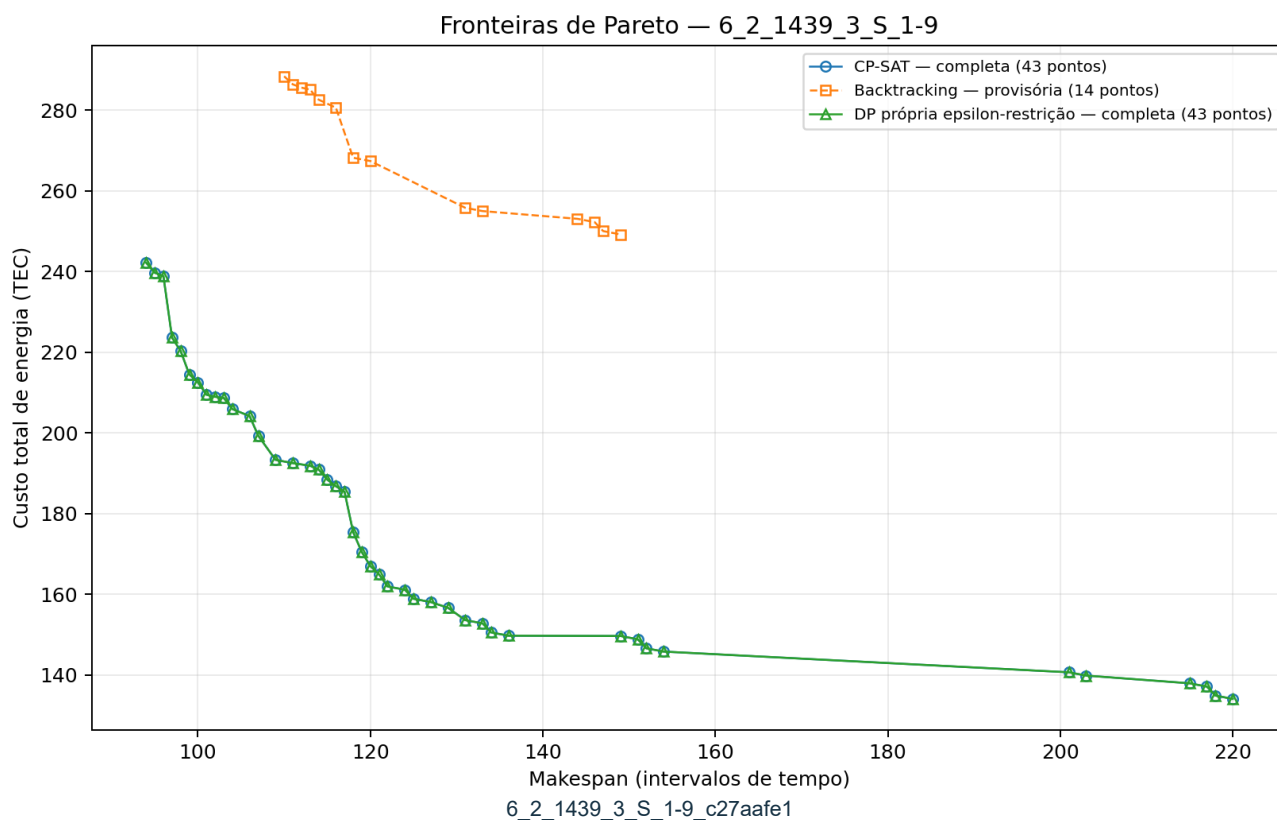
12 resultados congelados e 413 escalonamentos verificados no snapshot da auditoria: durações, jobs, horizonte, setups e custos racionais corretos.

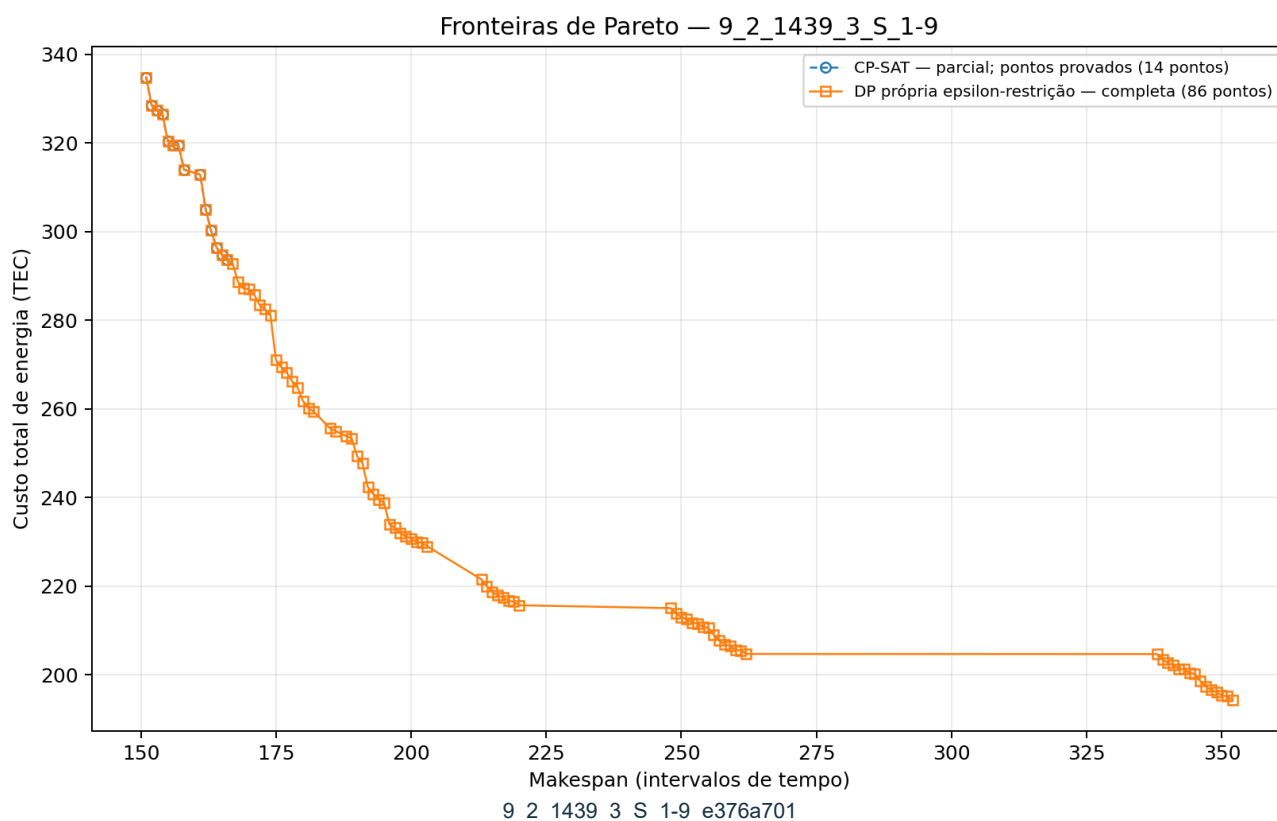
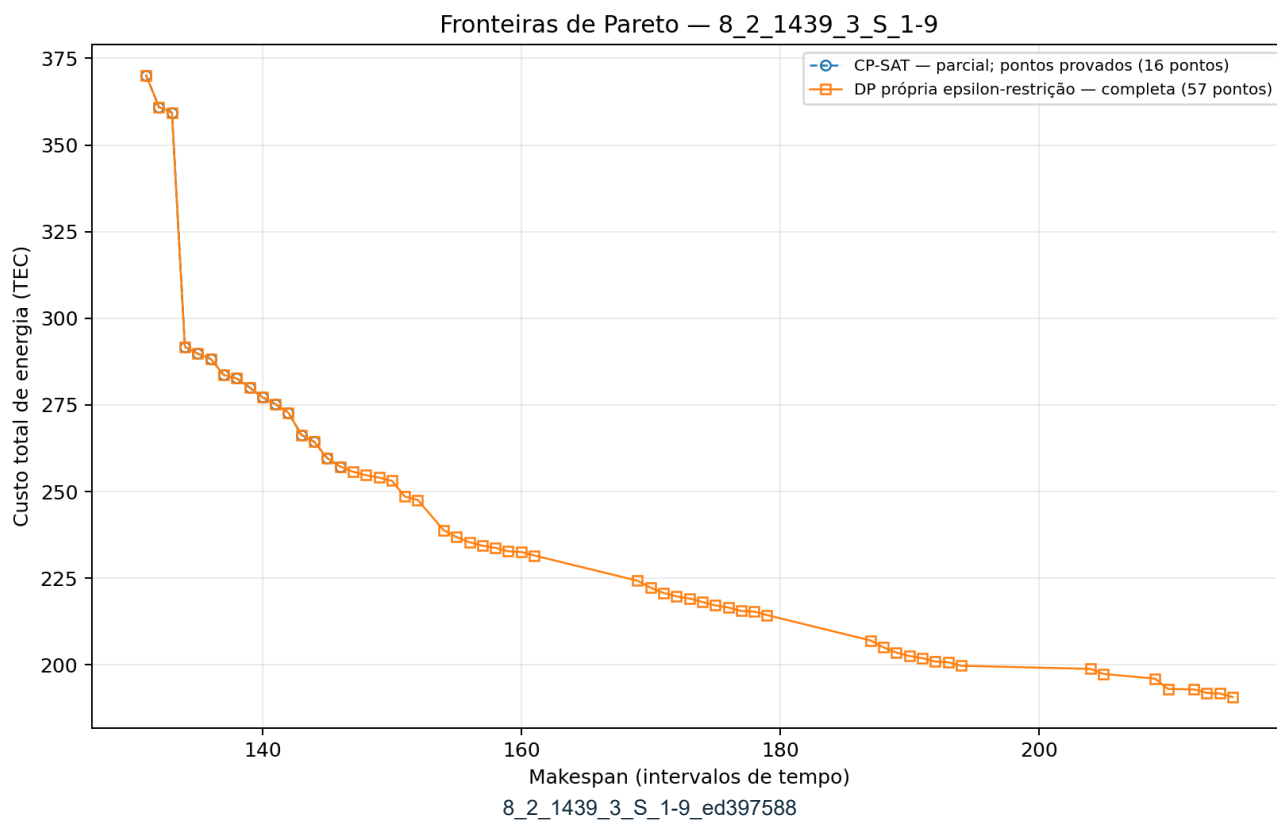
58 amostras de convergência com HV não decrescente nas duas buscas prolongadas. Cobertura de combinações não é cobertura da fronteira.

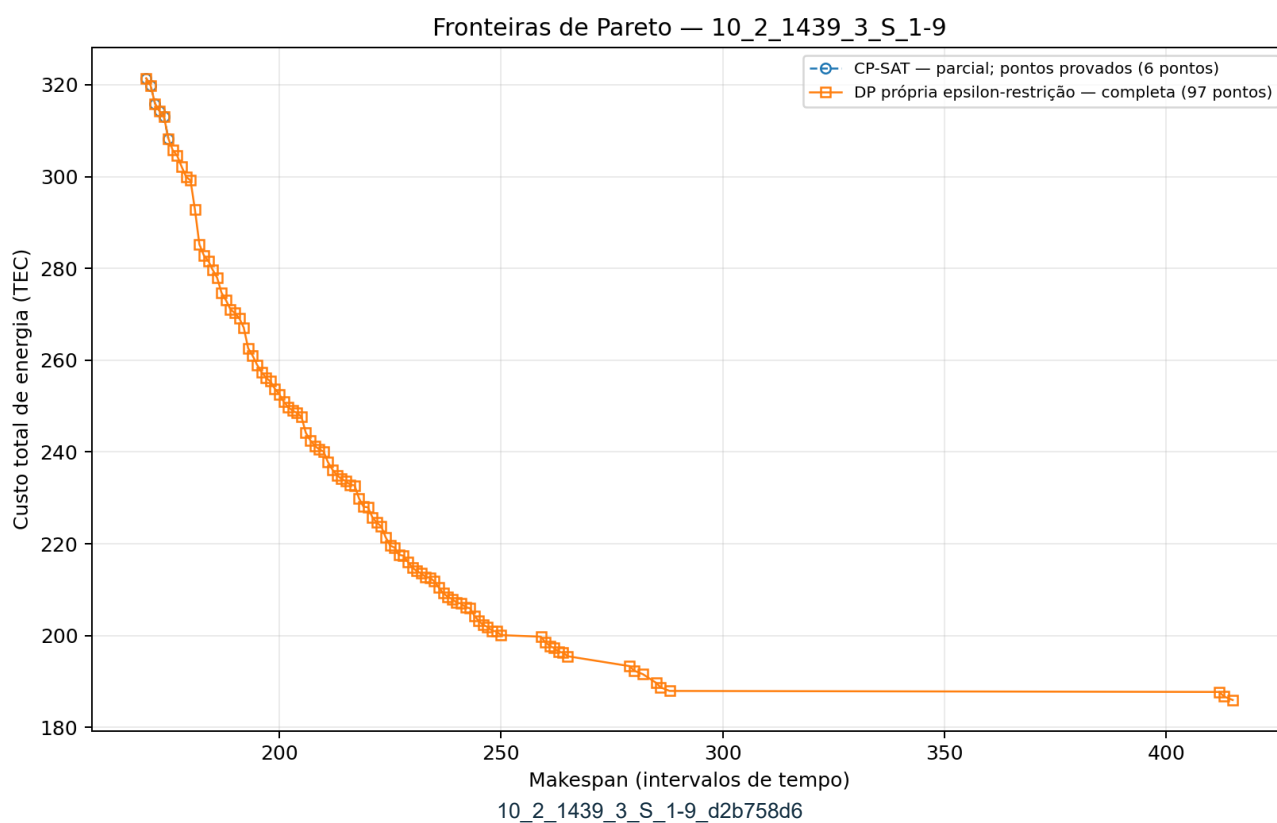
Ajustes editoriais incorporados: combinações cobertas por regras/dominância e coluna epsilon da DP, sem sugerir falta de prova CP-SAT.

A inspeção visual do PDF pertence à raiz; não faz parte desta nota experimental. Avanços posteriores ao snapshot são validados pelo script, não atribuídos retroativamente ao auditor.

10. Fronteiras encontradas







11. Makespan mínimo por DP própria

`src/exact-solutions/makespan_dp.py`

Módulo de otimização mono-objetivo escrito com biblioteca padrão. Fornece o primeiro epsilon da varredura. Não minimiza energia e não lê resultados de outros solvers.

Funções: 5. SHA-256: 9032f32191481219ae20...

11.1 minimum_makespan

`minimum_makespan(instance: Instance, *, max_jobs: int=16)`

Responsabilidade. Determinar o menor término possível de todas as tarefas e reconstruir uma solução que o atinge.

Funcionamento e contratos. Recebe uma Instance e um limite de tamanho `max_jobs`. Escolhe o modo de menor duração para cada tarefa/máquina, resolve as sequências de todos os subconjuntos e particiona os subconjuntos entre as máquinas. Máquinas vazias são permitidas. Se o ótimo excede o horizonte, a entrada é inviável sob esse horizonte.

Retorno e efeitos. Dicionário com `minimum_makespan`, `schedule`, `status`, `horizonte`, `hash` e `tempo`. Não grava arquivos por conta própria. Rejeita instâncias acima do limite de tarefas.

Referência: linha 17 do fonte identificado acima.

11.2 minimum_makespan.ending

`minimum_makespan.ending(i, subset, last)`

Responsabilidade. Calcular o menor tempo de uma sequência na máquina `i`, executando `subset` e terminando em `last`.

Funcionamento e contratos. A chave do cache é `(i, subset, last)`. Para uma tarefa isolada retorna sua duração. Nos demais casos testa cada possível predecessor: soma o melhor tempo do subconjunto anterior, o setup e a duração de `last`. `subset` é uma máscara de bits; remover `last` equivale a desligar seu bit.

Retorno e efeitos. Tempo inteiro ótimo para esse estado. `lru_cache` reaproveita estados repetidos durante a mesma chamada principal.

Referência: linha 34 do fonte identificado acima.

11.3 minimum_makespan.machine_time

`minimum_makespan.machine_time(i, subset)`

Responsabilidade. Obter a melhor sequência de um subconjunto em uma máquina, sem fixar a última tarefa.

Funcionamento e contratos. Recebe máquina e máscara do subconjunto. Consulta `ending` para cada tarefa que poderia ser a última e toma o mínimo. O conjunto vazio leva tempo zero, permitindo deixar uma máquina sem tarefas.

Retorno e efeitos. Menor tempo inteiro do subconjunto naquela máquina, armazenado em cache.

Referência: linha 43 do fonte identificado acima.

11.4 minimum_makespan.partition

`minimum_makespan.partition(count, subset)`

Responsabilidade. Distribuir as tarefas entre as primeiras count máquinas minimizando o maior término.

Funcionamento e contratos. Para uma máquina, todas as tarefas restantes vão para ela. Com mais máquinas, enumera o subconjunto atribuído à última máquina e resolve recursivamente o complemento. Compara o máximo entre os tempos dos dois lados. A identidade das máquinas é preservada, pois elas são não relacionadas.

Retorno e efeitos. Par (melhor makespan, subconjunto atribuído à última máquina). O segundo valor permite reconstruir a divisão.

Referência: linha 47 do fonte identificado acima.

11.5 main

`main()`

Responsabilidade. Executar o cálculo do makespan mínimo pela linha de comando.

Funcionamento e contratos. Lê um arquivo .dat ou uma pasta de instâncias. Por padrão usa data/input/set1. Invoca minimum_makespan para cada entrada e grava um JSON em data/output/makespan_dp. Entradas inválidas são relatadas pelo argparse.

Retorno e efeitos. Não retorna um resultado de otimização ao chamador; grava relatórios e imprime o resumo de cada instância.

Referência: linha 94 do fonte identificado acima.

12. Fronteira com código próprio

`src/exact-solutions/epsilon_exact.py`

Esta é a nova solução principal, sem SciPy, NumPy ou OR-Tools. Usa rótulos de programação dinâmica e visita cada epsilon inteiro. Label é uma estrutura imutável com end, energy, job, index e previous; previous encadeia o escalonamento para reconstrução.

Funções: 11. SHA-256: 141263c95cf566b94a29...

12.1 insert_label

`insert_label(front, candidate)`

Responsabilidade. Manter somente rótulos não dominados em um estado da programação dinâmica.

Funcionamento e contratos. Recebe uma lista ordenada e um candidato. Faz busca binária pelo término, verifica se um rótulo anterior já domina o candidato e remove rótulos posteriores que ele domina. Ao final, termos crescem e energias decrescem estritamente. A comparação usa inteiros, inclusive para empates.

Retorno e efeitos. True se inseriu o candidato; False se ele já era dominado. Altera a lista recebida. Não elimina rótulos de estados com últimas tarefas diferentes durante as transições.

Referência: linha 35 do fonte identificado acima.

12.2 next_cheaper

`next_cheaper(costs)`

Responsabilidade. Pré-calcular o próximo horário inicial que reduz estritamente o custo de uma tarefa em um modo fixo.

Funcionamento e contratos. Recebe os custos ordenados por início. Percorre a lista de trás para frente com uma pilha, retirando valores maiores ou iguais. O topo restante é o primeiro índice posterior com custo menor. A construção é linear: cada índice entra e sai da pilha no máximo uma vez.

Retorno e efeitos. Lista de índices. `len(costs)` funciona como sentinela quando não existe horário posterior mais barato. Exemplo: `[5,5,7,3,4,1]` gera `[3,3,3,5,5,6]`.

Referência: linha 56 do fonte identificado acima.

12.3 best_before

`best_before(front, epsilon)`

Responsabilidade. Selecionar o rótulo de menor energia que termina até epsilon.

Funcionamento e contratos. Recebe uma fronteira mantida por `insert_label`. Como os termos crescem e os custos decrescem, o último rótulo cujo término cabe no teto é o mais barato admissível. Uma busca binária encontra esse rótulo sem percorrer a lista inteira.

Retorno e efeitos. Um Label, ou None se nenhum término cabe no teto. A lista original não é alterada.

Referência: linha 68 do fonte identificado acima.

12.4 EnergyDP.__init__

```
EnergyDP.__init__(self, instance)
```

Responsabilidade. Preparar os dados tarifários e as alternativas usadas pela DP de energia.

Funcionamento e contratos. Recebe a instância, limita o método a 12 tarefas e gera opções máquina/modo/início com custos inteiros. Agrupa índices por máquina, tarefa e modo; calcula next_cheaper de cada grupo e mapas para reconstrução. Soma os menores custos individuais como limite inferior global.

Retorno e efeitos. Inicializa choices, scale, groups, lookup, lower_bound, contadores e ready=False. Ainda não há prova de otimalidade da energia.

Referência: linha 81 do fonte identificado acima.

12.5 EnergyDP.prepare

```
EnergyDP.prepare(self, *, deadline=None, max_transitions=None)
```

Responsabilidade. Construir integralmente as tabelas de sequências e energia de cada máquina.

Funcionamento e contratos. Parte da sequência vazia. Para cada subconjunto, última tarefa e rótulo, acrescenta cada tarefa restante e cada modo. Respeita o setup, considera o primeiro início permitido e apenas os inícios posteriores mais baratos. insert_label elimina estados dominados. Após concluir uma máquina, consolida os últimos jobs porque a etapa de partição não acrescentará mais tarefas àquela sequência.

Retorno e efeitos. True e ready=True quando todas as máquinas foram concluídas. False se o prazo ou max_transitions interromper a preparação. Atualiza frontiers e stats; tabelas incompletas nunca são usadas como prova.

Referência: linha 102 do fonte identificado acima.

12.6 EnergyDP.solve

```
EnergyDP.solve(self, epsilon)
```

Responsabilidade. Resolver exatamente a menor energia sob um teto de makespan, usando tabelas já completas.

Funcionamento e contratos. Exige ready=True e epsilon no horizonte. Escolhe o melhor rótulo de cada subconjunto/máquina que caiba em epsilon. A função interna partition testa as divisões das tarefas. Depois reconstrói índices seguindo os ponteiros previous e verifica unicidade, sobreposição, setups, tempo e energia.

Retorno e efeitos. ((makespan real, energia inteira), índices das opções por tarefa), ou None se o teto for inviável. Falhas de reconstrução geram AssertionError.

Referência: linha 161 do fonte identificado acima.

12.7 EnergyDP.solve.partition

EnergyDP.solve.partition(count, subset)

Responsabilidade. Minimizar a soma das energias de subconjuntos atribuídos a máquinas diferentes.

Funcionamento e contratos. O estado é (count, subset). Enumera o subconjunto da última máquina e combina seu custo com o ótimo do complemento nas demais máquinas. Subconjuntos vazios são válidos. Empates de energia são resolvidos pelo menor makespan. Usa cache durante a resolução de um epsilon.

Retorno e efeitos. Tripla (energia, makespan, subconjunto da última máquina). Um custo infinito identifica uma divisão inviável.

Referência: linha 171 do fonte identificado acima.

12.8 EnergyDP.indices_from_schedule

EnergyDP.indices_from_schedule(self, schedule)

Responsabilidade. Converter um escalonamento viável conhecido para a representação de índices usada pelos relatórios.

Funcionamento e contratos. Percorre registros com job, machine, mode e start. Consulta lookup para achar a opção correspondente de cada tarefa. O principal uso é representar a solução inicial da DP de makespan como candidato antes de provar o custo mínimo.

Retorno e efeitos. Tupla de índices, ordenada por tarefa. Não otimiza nem atribui prova de energia ao escalonamento recebido.

Referência: linha 210 do fonte identificado acima.

12.9 solve_front

solve_front(instance, *, output, total_seconds=300.0, epsilon_max=None, resume=False, restart=False, max_transitions=None)

Responsabilidade. Coordenar makespan mínimo, tabelas de energia, varredura unitária, persistência e retomada.

Funcionamento e contratos. Valida caminhos, orçamento, teto e compatibilidade por hash. Calcula a primeira solução pela DP de makespan, prepara a DP de energia e resolve cada epsilon inteiro até o teto. Atualiza o arquivo de Pareto com dominância exata. Se interromper antes de concluir as tabelas, preserva apenas um candidato viável; a retomada reconstrói as tabelas e continua pelos epsilons pendentes.

Retorno e efeitos. Relatório JSON em memória e em output. pareto_proven só é verdadeiro após todo o horizonte ou após prova de inviabilidade da instância. Nunca importa resultados CP-SAT/MILP.

Referência: linha 218 do fonte identificado acima.

12.10 solve_front.save

solve_front.save(reason)

Responsabilidade. Gravar de forma consistente o estado público da varredura própria.

Funcionamento e contratos. Acessa report, archive, energy e relógios do escopo externo. Determina se o intervalo solicitado e o horizonte inteiro foram concluídos. Reconstrói a fronteira com escalonamentos, registra estatísticas e tempo acumulado e chama atomic_json.

Retorno e efeitos. O relatório atualizado. A gravação substitui o JSON de destino atômicamente, mas não armazena as tabelas internas da DP.

Referência: linha 269 do fonte identificado acima.

12.11 main

`main()`

Responsabilidade. Disponibilizar a solução própria pela linha de comando.

Funcionamento e contratos. Aceita arquivo ou diretório, `--output-dir`, `--total-time-limit`, `--epsilon-max`, `--resume` e `--restart`. O padrão é a instância de seis tarefas e 300 segundos por execução. Um limite zero permite executar sem esse orçamento.

Retorno e efeitos. Grava um relatório por entrada em `data/output/epsilon_exact` e imprime estado, quantidade de pontos e motivo de parada.

Referência: linha 306 do fonte identificado acima.

13. Backtracking e podas seguras

src/exact-solutions/custom_exact.py

Solver próprio já existente e ampliado nesta conversa. O catálogo inclui todas as suas funções para contextualizar as novas podas. As decisões fixam posições de tarefas; máscaras de bits representam horários ainda permitidos.

Funções: 13. SHA-256: 339f9183da2507a8ec25...

13.1 Domains.__init__

```
Domains.__init__(self, instance: Instance, choices: tuple[tuple[Choice, ...], ...])
```

Responsabilidade. Organizar as opções e pré-calcular estruturas para propagação e limites inferiores.

Funcionamento e contratos. Recebe a instância e as opções de cada tarefa. Cria grupos máquina/modo, máscaras por custo, durações e ordenação por término. Calcula por Floyd-Warshall a menor separação possível entre pares, incluindo processamentos intermediários mínimos. Permitir qualquer intermediário mantém o limite otimista, mesmo se o setup direto puder ser reparado por inserção.

Retorno e efeitos. Inicializa initial, cost_masks, ordered_indices, lengths, durations, counts e separation. Não escolhe um escalonamento.

Referência: linha 26 do fonte identificado acima.

13.2 Domains.allowed

```
Domains.allowed(self, masks: list[list[int]], job: int, choice: Choice)
```

Responsabilidade. Consultar se uma opção ainda pertence ao domínio de uma tarefa.

Funcionamento e contratos. Obtém o grupo de máquina/modo da opção e testa o bit correspondente ao horário inicial. O estado dos domínios já incorpora decisões anteriores. Um bit desligado representa uma posição que não precisa ser explorada.

Retorno e efeitos. Booleano, sem alterar os domínios.

Referência: linha 73 do fonte identificado acima.

13.3 Domains.after

```
Domains.after(self, masks: list[list[int]], selected_job: int, choice: Choice,
               selected_indices: list[int])
```

Responsabilidade. Propagar uma alocação recém-escolhida sobre as tarefas ainda livres.

Funcionamento e contratos. Copia as máscaras para preservar o estado do pai. Na máquina escolhida, remove inícios que causariam sobreposição ou separação insuficiente em qualquer das duas ordens temporais. Usa os limites de separation, não um setup direto arbitrário que poderia ser reparado por uma tarefa intermediária.

Retorno e efeitos. Novas máscaras de domínio. Não altera a matriz recebida nem as decisões já selecionadas.

Referência: linha 77 do fonte identificado acima.

13.4 Domains.choose_job

```
Domains.choose_job(self, masks: list[list[int]], remaining: list[int])
```

Responsabilidade. Escolher qual tarefa livre será decidida no próximo nível da busca.

Funcionamento e contratos. Conta bits permitidos e seleciona a tarefa com menos alternativas. Em empate, prioriza maior duração mínima e depois o identificador. Essa regra altera a ordem de exploração; não remove soluções.

Retorno e efeitos. Índice de uma tarefa presente na lista remaining.

Referência: linha 96 do fonte identificado acima.

13.5 Domains.candidate_indices

```
Domains.candidate_indices(self, masks: list[list[int]], job: int)
```

Responsabilidade. Listar as opções ainda permitidas da tarefa escolhida.

Funcionamento e contratos. Percorre a ordem preparada no construtor e mantém somente posições aceitas por allowed. A lista prioriza termos menores e, nos empates, custos menores. Cada elemento é um índice na lista original de opções, permitindo salvar a busca sem copiar objetos Choice.

Retorno e efeitos. Lista ordenada de índices candidatos para um quadro da pilha.

Referência: linha 102 do fonte identificado acima.

13.6 Domains.nondominated_candidates

```
Domains.nondominated_candidates(self, masks, selected, job, candidates, archive)
```

Responsabilidade. Eliminar em blocos os candidatos cuja subárvore inteira é dominada por uma solução conhecida.

Funcionamento e contratos. Calcula objetivos otimistas do estado, retira o menor custo do job atual e substitui pelo custo de cada grupo de opções dele. Compara com os pontos do arquivo de Pareto. Máscaras removem inícios suficientemente tardios e caros para serem dominados. A poda é reaplicada quando a revisão do arquivo de Pareto muda.

Retorno e efeitos. Lista filtrada de candidatos ainda não visitados. Não usa soluções externas nem afirma que opções de um ramo incompleto sejam viáveis.

Referência: linha 106 do fonte identificado acima.

13.7 Domains.optimistic_objectives

```
Domains.optimistic_objectives(self, masks: list[list[int]], selected_indices: list[int])
```

Responsabilidade. Calcular limites inferiores seguros de makespan e energia para um estado parcial.

Funcionamento e contratos. Soma os custos fixados e os mínimos dos jobs livres. O término considera decisões fixadas e menores termos individuais. Acrescenta limites de capacidade: carga mínima total dividida pelo número de máquinas e carga obrigatória por máquina. Disputas e setups adicionais ignorados só tornam esses limites menores, preservando a segurança.

Retorno e efeitos. Par (limite de makespan, limite de energia), ou None se um domínio está vazio ou a carga mínima excede o horizonte.

Referência: linha 136 do fonte identificado acima.

13.8 irreparable_setup

```
irreparable_setup(instance: Instance, domains: Domains, selected_indices: list[int])
```

Responsabilidade. Identificar conflitos de setup entre decisões já fixadas que nenhuma inserção futura pode reparar.

Funcionamento e contratos. Ordena os jobs escolhidos por máquina. Para cada adjacência com folga menor que o setup direto, verifica se a folga é também menor que o menor processamento de qualquer job restante naquela máquina. Nesse caso, nem uma tarefa intermediária cabe. Conflitos potencialmente reparáveis são mantidos.

Retorno e efeitos. True quando existe conflito seguramente irreparável; False não garante viabilidade, apenas ausência dessa prova de impossibilidade.

Referência: linha 183 do fonte identificado acima.

13.9 search

```
search(instance: Instance, *, checkpoint: Path, output: Path, time_limit: float |  
        None=10.0, progress_interval: float=2.0, checkpoint_interval: float=5.0, max_decisions:  
        int | None=None, resume: bool=False, restart: bool=False)
```

Responsabilidade. Percorrer a árvore de decisões, aplicar podas, atualizar a fronteira e salvar checkpoints.

Funcionamento e contratos. Recebe instância, caminhos e limites. Usa pilha explícita de quadros para permitir retomada. Reconstrói domínios do checkpoint, valida hash e versão, propaga decisões e calcula limites. Nas folhas verifica setups e insere pares não dominados. Um contador acompanha as combinações cobertas por exploração e por poda.

Retorno e efeitos. Relatório completo ou provisório. Grava output e checkpoint periodicamente e ao parar. Só prova a fronteira ao esgotar a pilha e conferir a cobertura total.

Referência: linha 205 do fonte identificado acima.

13.10 search.record

```
search.record(reason: str, combinations: int)
```

Responsabilidade. Contabilizar quantas combinações foram dispensadas por uma poda.

Funcionamento e contratos. Recebe o motivo e o número de combinações da subárvore. Incrementa o contador específico e covered_combinations. Esses números contam combinações brutas cobertas, não tempo economizado nem nós realmente visitados.

Retorno e efeitos. Sem retorno; modifica stats do escopo de search.

Referência: linha 280 do fonte identificado acima.

13.11 search.save

```
search.save(complete: bool, reason: str | None)
```

Responsabilidade. Persistir o checkpoint retomável e o relatório legível do backtracking.

Funcionamento e contratos. Registra quadros, decisões, revisão do Pareto, índices dos pontos, estatísticas e tempo. Converte a fronteira para escalonamentos completos. A versão 2 evita retomar checkpoints gerados antes das alterações de propagação e candidatos.

Retorno e efeitos. Dicionário do relatório após duas gravações atômicas: estado interno e saída pública.

Referência: linha 284 do fonte identificado acima.

13.12 search.show_progress

```
search.show_progress()
```

Responsabilidade. Mostrar o avanço da busca no terminal.

Funcionamento e contratos. Calcula a porcentagem do produto cartesiano coberta e imprime decisões, folhas viáveis, tamanho do Pareto e tempo acumulado. A porcentagem não é uma previsão linear do tempo restante, porque subárvores têm custos distintos.

Retorno e efeitos. Sem retorno; imprime uma linha de acompanhamento.

Referência: linha 304 do fonte identificado acima.

13.13 main

```
main(argv: list[str] | None=None)
```

Responsabilidade. Interpretar argumentos e executar o backtracking em um arquivo ou pasta.

Funcionamento e contratos. Permite limitar tempo e decisões, controlar intervalos de gravação, retomar ou reiniciar. --time-limit 0 remove o limite temporal. A parada por teclado interrompe o lote. Entradas ou checkpoints inválidos são relatados pelo parser.

Retorno e efeitos. Código de saída zero ao concluir a rotina normal. Relatórios e checkpoints ficam em data/output/custom_exact por padrão.

Referência: linha 400 do fonte identificado acima.

14. Leitura e desenho das fronteiras

`src/plot_pareto.py`

Ferramenta de apresentação com Matplotlib. Não executa métodos de otimização. Separa instâncias por nome e hash; preserva a diferença entre fronteira completa, aproximação provisória e candidato sem prova.

Funções: 5. SHA-256: a2902e5fab4a576629b8...

14.1 point_values

`point_values(point, scale=1)`

Responsabilidade. Ler um par de objetivos nos diferentes formatos JSON usados pelo projeto.

Funcionamento e contratos. Aceita `tec_exact`, numerador/denominador do baseline, ponto inteiro com escala ou TEC decimal de apresentação. Converte a energia para Fraction, para que filtragem e empates não dependam de arredondamento binário.

Retorno e efeitos. Par (`makespan`, energia racional). A escala é necessária para formatos que guardam apenas energia inteira.

Referência: linha 20 do fonte identificado acima.

14.2 nondominated

`nondominated(points)`

Responsabilidade. Remover duplicatas e pontos dominados de uma série de resultados.

Funcionamento e contratos. Ordena os pares por `makespan` e energia. Mantém apenas custos estritamente menores que o melhor já visto. No mesmo `makespan`, o ponto mais barato aparece primeiro. O procedimento filtra um conjunto recebido, mas não prova que ele seja a fronteira global.

Retorno e efeitos. Lista ordenada de pares não dominados daquele conjunto.

Referência: linha 32 do fonte identificado acima.

14.3 load_reports

`load_reports(paths)`

Responsabilidade. Descobrir e selecionar relatórios existentes para o gráfico.

Funcionamento e contratos. Recebe arquivos ou pastas, procura JSONs e ignora checkpoints. Decodifica fronteiras e candidatos; entradas sem pontos são omitidas. Para método, nome e hash iguais, prefere um resultado completo e, depois, o arquivo gravado mais recentemente. Hashes distintos nunca são fundidos.

Retorno e efeitos. Lista de séries com método, origem, status e pontos. Arquivos ilegíveis geram uma mensagem e são ignorados.

Referência: linha 41 do fonte identificado acima.

14.4 plot_reports

```
plot_reports(paths, output_dir, show=False)
```

Responsabilidade. Gerar um gráfico de comparação para cada instância identificada.

Funcionamento e contratos. Agrupa as séries, escolhe cores e marcadores e identifica resultados completos ou parciais na legenda. Candidatos sem prova usam cruzeiros. O backend Agg permite salvar PNG sem janela. --show usa a janela do Matplotlib. As linhas entre pontos são apenas guias visuais.

Retorno e efeitos. Lista dos caminhos PNG gerados. Cria a pasta de saída e imprime as fontes utilizadas.

Referência: linha 82 do fonte identificado acima.

14.5 main

```
main()
```

Responsabilidade. Executar a geração de gráficos a partir da linha de comando.

Funcionamento e contratos. Aceita caminhos opcionais, diretório de saída e --show. Sem caminhos, lê data/baselines e data/output. Dessa forma inclui tanto resultados históricos como execuções próprias novas, sem iniciar buscas demoradas.

Retorno e efeitos. Imprime os caminhos dos gráficos; por padrão os salva em data/output/plots.

Referência: linha 132 do fonte identificado acima.

15. MILP anterior: referência de biblioteca

`src/exact-solutions/epsilon_milp.py`

Implementação criada anteriormente nesta conversa e preservada para comparação. Usa SciPy/HiGHS; não atende ao requisito da nova solução própria e não é importada por `epsilon_exact.py`. É documentada porque faz parte dos códigos produzidos.

Funções: 8. SHA-256: e600222d4f6219476276...

15.1 `affine_segments`

`affine_segments(costs)`

Responsabilidade. Comprimir uma tabela de custo por início em segmentos afins exatos sobre horários inteiros.

Funcionamento e contratos. Recebe custos ordenados por início. Agrupa diferenças consecutivas iguais; cada grupo tem limites, inclinação e intercepto. Um segmento com um único início usa inclinação zero. A partição cobre todos os horários inteiros sem aproximar tarifas.

Retorno e efeitos. Gerador de tuplas (início mínimo, início máximo, inclinação, intercepto).

Referência: linha 25 do fonte identificado acima.

15.2 `EnergyModel.__init__`

`EnergyModel.__init__(self, instance)`

Responsabilidade. Preparar alternativas, custos e segmentos para o modelo MILP.

Funcionamento e contratos. Gera Choices, encontra um limite inferior global e reduz os coeficientes de energia pelo máximo divisor comum. Agrupa as opções por tarefa, máquina e modo, calcula segmentos e prepara mapas de reconstrução.

Retorno e efeitos. Inicializa os dados do modelo; ainda não chama o solver.

Referência: linha 38 do fonte identificado acima.

15.3 `EnergyModel.solve`

`EnergyModel.solve(self, epsilon, seconds=60.0, cost_cap=None)`

Responsabilidade. Construir e resolver um MILP de energia para um epsilon.

Funcionamento e contratos. Cria variáveis inteiras de tempos, segmentos e arcos sucessores. Impõe alocação única, caminhos por máquina e setups com big-M. Usa SciPy milp com gap relativo zero. Reconstrói a resposta e reavalia custos/restrições com inteiros; o bound numérico também deve excluir o custo inteiro imediatamente inferior.

Retorno e efeitos. Metadados do solver e um candidato, quando disponível. Limite de tempo não é tratado como otimalidade. SciPy usa float64 e tolerâncias; esta versão não é a nova solução do zero.

Referência: linha 56 do fonte identificado acima.

15.4 EnergyModel.solve.variable

```
EnergyModel.solve.variable(bound, cost=0)
```

Responsabilidade. Acrescentar uma variável ao modelo esparso.

Funcionamento e contratos. Recebe limite superior e coeficiente de custo. Adiciona ambos às listas locais. O limite inferior zero e a integralidade são definidos posteriormente para todas as variáveis na chamada milp.

Retorno e efeitos. Índice inteiro da nova variável, usado como coluna nas restrições.

Referência: linha 67 do fonte identificado acima.

15.5 EnergyModel.solve.constraint

```
EnergyModel.solve.constraint(terms, low=-np.inf, high=np.inf)
```

Responsabilidade. Acrescentar uma restrição linear à matriz esparsa.

Funcionamento e contratos. Recebe pares (coluna, coeficiente) e limites inferior/superior. Ignora coeficientes zero, registra os demais em formato de coordenadas e guarda os limites da linha. Igualdades usam limites iguais.

Retorno e efeitos. Sem retorno; modifica listas locais que depois formam LinearConstraint.

Referência: linha 73 do fonte identificado acima.

15.6 solve_front

```
solve_front(instance, *, output, seconds_per_solve=60.0, total_seconds=300.0,  
            epsilon_max=None, resume=False, restart=False)
```

Responsabilidade. Varrer epsilons com chamadas MILP e registrar apenas os pontos cuja otimalidade foi aceita.

Funcionamento e contratos. Calcula o mínimo makespan por DP, constrói EnergyModel e resolve tetos consecutivos. O custo ótimo anterior limita o próximo problema. Se o custo atinge o limite inferior global, certifica os epsilons restantes sem novas chamadas. Interrupções preservam o epsilon pendente e um candidato separado.

Retorno e efeitos. Relatório e JSON em output, incluindo sweep, iterations, front e status. O relatório é compatível com plot_pareto.

Referência: linha 179 do fonte identificado acima.

15.7 solve_front.save

```
solve_front.save(reason)
```

Responsabilidade. Atualizar e salvar o relatório da varredura MILP.

Funcionamento e contratos. Determina conclusão da faixa e do horizonte inteiro, registra tempo e motivo de parada e converte o arquivo interno de Pareto para escalonamentos. A gravação usa atomic_json.

Retorno e efeitos. O relatório atualizado, sem guardar o estado interno do solver HiGHS.

Referência: linha 221 do fonte identificado acima.

15.8 main

`main()`

Responsabilidade. Expor a alternativa MILP pela linha de comando.

Funcionamento e contratos. Aceita entrada, limites por chamada e por execução, teto opcional e flags de retomada/reinício. Importações de SciPy ocorrem apenas quando a solução MILP é executada. A nova DP própria usa outro arquivo e outro diretório de resultados.

Retorno e efeitos. Relatórios em `data/output/epsilon_milp` e resumo no terminal.

Referência: linha 272 do fonte identificado acima.

16. Testes do backtracking

tests/test_custom_exact.py

Testes existentes e ajustados para as novas podas. Todos usam apenas a biblioteca padrão e o oráculo de enumeração exaustiva em entradas pequenas.

Funções: 3. SHA-256: a333f7a1c764b19c10a2...

16.1 CustomExactTest.test_random_small_instances_match_oracle

CustomExactTest.test_random_small_instances_match_oracle(self)

Responsabilidade. Comparar a busca própria com enumeração exaustiva em seis entradas aleatórias reproduzíveis.

Funcionamento e contratos. Usa sementes fixas, três tarefas, duas máquinas e dois modos. Gera dados temporários, executa a busca sem limite e compara exatamente a fronteira. Verifica também se o contador de combinações cobertas coincide com o produto cartesiano total.

Retorno e efeitos. Passa ou lança falha de unittest. Arquivos temporários são descartados automaticamente.

Referência: linha 18 do fonte identificado acima.

16.2 CustomExactTest.test_complete_front_matches_oracle

CustomExactTest.test_complete_front_matches_oracle(self)

Responsabilidade. Verificar as seis fixtures com características específicas do problema.

Funcionamento e contratos. Executa o solver até terminar e compara makespan e energia inteira com brute_front. Exige status completo, pareto_proven verdadeiro e cobertura de todas as combinações. Inclui o caso em que inserir uma tarefa repara um setup direto.

Retorno e efeitos. Validação de igualdade de fronteiras pequenas; não mede escalabilidade nas entradas grandes.

Referência: linha 44 do fonte identificado acima.

16.3 CustomExactTest.test_resume_preserves_search_and_empty_start

CustomExactTest.test_resume_preserves_search_and_empty_start(self)

Responsabilidade. Confirmar que interromper e retomar mantém a busca equivalente à execução contínua.

Funcionamento e contratos. Interrompe após três decisões, quando já existe um ponto provisório. Retoma o checkpoint e compara a fronteira e todas as estatísticas com uma execução nova. O limite foi reduzido de dez para três porque, com as novas podas, esse caso termina em seis decisões.

Retorno e efeitos. Falha se a retomada perder trabalho, repetir contagem ou produzir resultado diferente.

Referência: linha 58 do fonte identificado acima.

17. Testes de makespan e do MILP anterior

tests/test_epsilon_milp.py

Contém testes próprios da DP de makespan e testes opcionais da versão de biblioteca. A classe MILP é ignorada quando SciPy não está instalado; isso não impede testar os métodos próprios.

Funções: 6. SHA-256: d86929d351aad08fb5d0...

17.1 MakespanDPTTest.test_optimum_and_schedule

MakespanDPTTest.test_optimum_and_schedule(self)

Responsabilidade. Comparar o mínimo makespan e validar a sequência reconstruída.

Funcionamento e contratos. Para cada fixture, compara o resultado com o menor makespan da fronteira exaustiva. Converte o escalonamento em Choices e verifica ausência de sobreposição, setups e término final.

Retorno e efeitos. Confirma o valor e a viabilidade do representante, em vez de testar somente um número.

Referência: linha 19 do fonte identificado acima.

17.2 MakespanDPTTest.test_multiple_machines_and_infeasible_horizon

MakespanDPTTest.test_multiple_machines_and_infeasible_horizon(self)

Responsabilidade. Cobrir a partição geral em três máquinas e uma entrada que não cabe no horizonte.

Funcionamento e contratos. Amplia uma fixture para três máquinas e compara o mínimo com o oráculo. Depois aumenta os tempos de processamento e exige que a DP identifique a inviabilidade do horizonte.

Retorno e efeitos. Falha se a partição funcionar apenas para duas máquinas ou se a inviabilidade for tratada como ótimo viável.

Referência: linha 31 do fonte identificado acima.

17.3 EpsilonMILPTest.setUpClass

EpsilonMILPTest.setUpClass(cls)

Responsabilidade. Verificar a disponibilidade da biblioteca opcional antes dos testes MILP.

Funcionamento e contratos. Tenta importar scipy.optimize.milp. A ausência gera SkipTest para essa classe, deixando os testes de biblioteca padrão disponíveis.

Retorno e efeitos. Sem retorno normal; pode marcar a classe como ignorada.

Referência: linha 43 do fonte identificado acima.

17.4 EpsilonMILPTest.test_front_and_every_epsilon_match_oracle

EpsilonMILPTest.test_front_and_every_epsilon_match_oracle(self)

Responsabilidade. Comparar a fronteira e o ótimo de cada teto do MILP com o oráculo.

Funcionamento e contratos. Executa as seis fixtures, exige conclusão e confere todos os epsilons até o horizonte. Para cada linha de sweep, calcula a menor energia do oráculo que respeita o teto. Confere também a lista final de pontos.

Retorno e efeitos. Validação dos platôs e dos pontos de melhoria, não apenas dos extremos.

Referência: linha 49 do fonte identificado acima.

17.5 EpsilonMILPTest.test_resume_and_timeout_candidate

EpsilonMILPTest.test_resume_and_timeout_candidate(self)

Responsabilidade. Testar retomada de uma faixa parcial e tratamento de candidato sem prova.

Funcionamento e contratos. Compara execução parcial retomada com execução completa. Em seguida simula um retorno do solver por limite de tempo, mantendo uma solução viável, e exige que candidate seja preservado sem inserir o ponto em front nem avançar epsilon.

Retorno e efeitos. Falha se o programa confundir viabilidade com otimalidade.

Referência: linha 62 do fonte identificado acima.

17.6 EpsilonMILPTest.test_affine_compression_preserves_every_integer_cost

EpsilonMILPTest.test_affine_compression_preserves_every_integer_cost(self)

Responsabilidade. Garantir que a compressão em segmentos não altera custos.

Funcionamento e contratos. Reconstrói o custo de cada horário inteiro a partir de inclinação e intercepto e compara com todas as opções originais, para cada tarefa/máquina/modo das fixtures.

Retorno e efeitos. Validação exata da representação tarifária usada no modelo MILP.

Referência: linha 82 do fonte identificado acima.

18. Testes da solução inteiramente própria

tests/test_epsilon_exact.py

Testes sem bibliotecas de otimização. Cobrem cada epsilon, estruturas de dominância, tarifas incomuns, mais máquinas e estados interrompidos.

Funções: 3. SHA-256: db5d6f44595989c109fd...

18.1 EpsilonExactTest.test_every_epsilon_matches_exhaustive_front

EpsilonExactTest.test_every_epsilon_matches_exhaustive_front(self)

Responsabilidade. Validar o algoritmo próprio para todos os limites de pequenas instâncias.

Funcionamento e contratos. Usa as seis fixtures e três variantes: ponta gratuita com tarifa externa maior, energia totalmente zero e três máquinas. Compara a fronteira e cada valor de sweep com brute_front. Esses casos verificam que a eliminação de horários não presume que ponta seja sempre mais cara.

Retorno e efeitos. Igualdade exata de energia inteira e makespan nas instâncias verificadas.

Referência: linha 18 do fonte identificado acima.

18.2 EpsilonExactTest.test_interruption_and_resume_do_not_claim_unproven_points

EpsilonExactTest.test_interruption_and_resume_do_not_claim_unproven_points(self)

Responsabilidade. Garantir que preparação incompleta não gere pontos supostamente provados.

Funcionamento e contratos. Interrompe antes da primeira transição da DP de energia. Exige front vazio e candidato preservado. Retoma para um teto parcial e depois para todo o horizonte; compara a fronteira e os valores de energia com uma execução nova.

Retorno e efeitos. Valida persistência, retomada e distinção entre candidato e solução ótima por epsilon.

Referência: linha 36 do fonte identificado acima.

18.3 EpsilonExactTest.test_front_helpers_and_infeasible_horizon

EpsilonExactTest.test_front_helpers_and_infeasible_horizon(self)

Responsabilidade. Verificar saltos tarifários, manutenção de rótulos e pré-condições da solução.

Funcionamento e contratos. Confere next_cheaper em uma série com empates, insere rótulos dominados e empatados e consulta best_before. Exige erro ao resolver antes de preparar tabelas. Por fim cria uma instância que não cabe no horizonte e verifica a prova de inviabilidade.

Retorno e efeitos. Cobertura das regras auxiliares que sustentam a poda e dos estados sem solução.

Referência: linha 52 do fonte identificado acima.

19. Gerador desta documentação

`docs/build_function_guide.py`

Ferramenta de documentação com ReportLab. Lê notas revisadas, confere cobertura usando AST e gera um PDF com sumário e resultados locais. O arquivo `function_notes.json` é a fonte textual editável das explicações.

Funções: 5. SHA-256: 6f517ae3a27c99b8c75d...

19.1 walk_functions

`walk_functions(node, prefix='')`

Responsabilidade. Inventariar todas as funções nomeadas de um arquivo Python.

Funcionamento e contratos. Percorre recursivamente a árvore sintática. Acrescenta o prefixo de classes e funções externas, identificando métodos e funções internas sem confundir nomes iguais em escopos diferentes. Expressões lambda não são funções nomeadas e são explicadas no comportamento da função que as contém.

Retorno e efeitos. Lista de tuplas (nome qualificado, assinatura dos argumentos, linha de definição).

Referência: linha 26 do fonte identificado acima.

19.2 page_decoration

`page_decoration(canvas, doc)`

Responsabilidade. Padronizar a identificação visual de cada página.

Funcionamento e contratos. Recebe canvas e documento do ReportLab, salva o estado gráfico e desenha cabeçalho, linha, rodapé e número da página. Restaura o estado para não alterar os elementos de conteúdo.

Retorno e efeitos. Sem retorno; desenha diretamente na página atual.

Referência: linha 39 do fonte identificado acima.

19.3 FunctionGuide.afterFlowable

`FunctionGuide.afterFlowable(self, flowable)`

Responsabilidade. Registrar os capítulos no sumário e na navegação do PDF.

Funcionamento e contratos. É chamado pelo ReportLab após posicionar cada elemento. Quando o elemento tem estilo Chapter, registra o título, a página e um marcador interno. multiBuild repete a composição até estabilizar as páginas do sumário.

Retorno e efeitos. Sem retorno; emite notificações de sumário e marcadores do documento.

Referência: linha 53 do fonte identificado acima.

19.4 append_markdown

`append_markdown(story, source, styles)`

Responsabilidade. Converter a análise Markdown para elementos de apresentação do PDF.

Funcionamento e contratos. Identifica capítulos, parágrafos e tabelas. Escapa HTML, repete cabeçalhos de tabelas e quebra inteiros longos para evitar ultrapassar a célula. Cada capítulo inicia nova página.

Retorno e efeitos. Acrescenta flowables à lista story; não altera os resultados de otimização.

Referência: linha 63 do fonte identificado acima.

19.5 build_pdf

`build_pdf()`

Responsabilidade. Gerar o relatório completo e documentar cada função dos arquivos catalogados.

Funcionamento e contratos. Combina dois catálogos de notas, inventaria funções pelo AST e rejeita entradas ausentes ou obsoletas. Usa hashes reais, fontes Arial/Consolas, sumário estável, análise, avaliações, gráficos e responsabilidades/contratos/retornos por função.

Retorno e efeitos. Grava `output/pdf/relatorio_completo_metodos_exatos.pdf`. Requer ReportLab somente para apresentação; o catálogo não é solver.

Referência: linha 103 do fonte identificado acima.

20. Modelo compartilhado e persistência

`src/exact-solutions/problem.py`

Convenções, parser, avaliação racional, opções individuais, dominância e gravação atômica usadas pelos três métodos.

Funções: 13. SHA-256: `adfbe9910f5dcfb9d12b...`

20.1 Instance.horizon

`Instance.horizon(self)`

Responsabilidade. Calcular o limite de término da instância.

Funcionamento e contratos. Multiplica `days` por `slots_per_day`, que já incorpora `hl+1`. Não infere o número de dias pelo nome do arquivo.

Retorno e efeitos. Inteiro `H`; 1440 em `set1` e vários dias em `set2`.

Referência: linha 39 do fonte identificado acima.

20.2 Instance.duration

`Instance.duration(self, job: int, machine: int, mode: int)`

Responsabilidade. Calcular a duração inteira de um job em máquina e modo.

Funcionamento e contratos. Divide `processing` pela velocidade como `Fraction` e arredonda para cima. Essa convenção evita duração insuficiente causada por truncamento ou float.

Retorno e efeitos. Inteiro positivo usado nas restrições e no custo.

Referência: linha 42 do fonte identificado acima.

20.3 read_instance

`read_instance(path: Path | str)`

Responsabilidade. Ler e validar o formato DAT em ordem definida.

Funcionamento e contratos. Resolve o caminho, calcula hash dos bytes, consome dimensões, tarifas, períodos, modos, potência, processamento e setups. Verifica positividade, intervalos por dia e ausência de tokens adicionais.

Retorno e efeitos. Instance imutável; `ValueError` para dados inconsistentes.

Referência: linha 58 do fonte identificado acima.

20.4 read_instance.take

`read_instance.take(key: str, count: int, convert)`

Responsabilidade. Consumir uma seção do parser.

Funcionamento e contratos. Confere a palavra-chave e quantidade, converte cada token com int ou `Fraction` e avança a posição compartilhada. Não aceita campos fora de ordem ou incompletos.

Retorno e efeitos. Tupla de valores; modifica somente o cursor interno.

Referência: linha 64 do fonte identificado acima.

20.5 energy_scale

`energy_scale(instance: Instance)`

Responsabilidade. Representar o custo exatamente por inteiros escalados.

Funcionamento e contratos. Multiplica potência, fator de modo, duração horária por slot e tarifa. O MMC dos denominadores permite preços inteiros sem mudar relações de dominância.

Retorno e efeitos. Escala e preços por máquina/modo, usados como energia/escala.

Referência: linha 114 do fonte identificado acima.

20.6 generate_choices

`generate_choices(instance: Instance, prices: tuple[tuple[int, int, int, int], ...])`

Responsabilidade. Gerar uma escolha individual para cada máquina, modo e início válido.

Funcionamento e contratos. Para cada job enumera start de zero a H-duração, incluindo o último permitido. Avalia sobreposição com ponta e custo exato. A ordem fixa permite persistir índices no checkpoint.

Retorno e efeitos. Tupla de opções por job. Se um job não cabe isoladamente, lança `ValueError`; CP-SAT/backtracking não publicam prova JSON nesse caso.

Referência: linha 132 do fonte identificado acima.

20.7 overlaps

`overlaps(choice: Choice, selected: list[Choice])`

Responsabilidade. Detectar processamento simultâneo na mesma máquina.

Funcionamento e contratos. Usa intervalos semiabertos: start de cada escolha deve ser menor que end da outra para haver interseção. Jobs em máquinas diferentes são independentes.

Retorno e efeitos. Booleano; não verifica setup.

Referência: linha 150 do fonte identificado acima.

20.8 has_valid_setups

`has_valid_setups(instance: Instance, selected: list[Choice])`

Responsabilidade. Validar setups entre consecutivos da sequência final.

Funcionamento e contratos. Agrupa por máquina, ordena por início e exige `start_next >= end_previous+setup`. Uma adjacência parcial pode mudar com inserções futuras, por isso não basta rejeitar qualquer par parcial pelo setup direto.

Retorno e efeitos. Booleano da regra de sequência.

Referência: linha 160 do fonte identificado acima.

20.9 dominates_or_equals

```
dominates_or_equals(a: tuple[int, int], b: tuple[int, int])
```

Responsabilidade. Comparar um par de objetivos com outro para minimização.

Funcionamento e contratos. Retorna verdadeiro se ambas as coordenadas de a não excedem as de b. A igualdade também permite rejeitar duplicatas.

Retorno e efeitos. Booleano sem tolerância de float.

Referência: linha 173 do fonte identificado acima.

20.10 insert_pareto

```
insert_pareto(archive: list[tuple[tuple[int, int], tuple[int, ...]]], point: tuple[int, int], choice_indices: tuple[int, ...])
```

Responsabilidade. Atualizar o arquivo de pontos não dominados encontrados.

Funcionamento e contratos. Rejeita um ponto dominado ou igual a qualquer existente; remove pontos dominados pelo novo; guarda índices que reconstruirão o schedule e ordena por objetivos.

Retorno e efeitos. Booleano indicando mudança; altera archive em memória.

Referência: linha 177 do fonte identificado acima.

20.11 atomic_json

```
atomic_json(path: Path, value: dict)
```

Responsabilidade. Gravar JSON de maneira atômica e resistente a bloqueio transitório do Windows.

Funcionamento e contratos. Escreve temporário, flush e fsync, depois os.replace. PermissionError no Windows permite até 12 tentativas com espera limitada (~3,62s no máximo). Nunca apaga o destino anterior antes da troca; negação permanente é propagada. Pressupõe um escritor por arquivo.

Retorno e efeitos. Grava o resultado ou mantém a versão anterior e lança erro. Retry não altera validade matemática dos resultados.

Referência: linha 188 do fonte identificado acima.

20.12 archive_output

```
archive_output(instance: Instance, choices: tuple[tuple[Choice, ...], ...], scale: int, archive: list[tuple[tuple[int, int], tuple[int, ...]]])
```

Responsabilidade. Converter pares e índices em relatórios legíveis e exatos.

Funcionamento e contratos. Reconstrói schedule e x_nonzero, calcula TEC como Fraction, preserva tec_exact e apresenta normalização por H e max_cost. As coordenadas começam em zero.

Retorno e efeitos. Lista de pontos com objetivos, frações, normalizadores e jobs/máquinas/modos/inícios/términos.

Referência: linha 207 do fonte identificado acima.

20.13 overlaps_any

`overlaps_any(selected: list[Choice])`

Responsabilidade. Verificar se qualquer par de escolhas sobrepõe processamento.

Funcionamento e contratos. Chama `overlaps` para cada posição em relação às anteriores. É usado na verificação de reconstruções.

Retorno e efeitos. Booleano; complexidade quadrática no número de jobs.

Referência: linha 232 do fonte identificado acima.

21. Método de biblioteca: CP-SAT

src/exact-solutions/cpsat.py

OR-Tools modela intervalos e circuitos; a enumeração lexicográfica certifica pontos e distingue candidatos sem prova.

Funções: 8. SHA-256: 50258d6e978f4ff49740...

21.1 ortools_module

ortools_module()

Responsabilidade. Carregar a dependência opcional de otimização.

Funcionamento e contratos. Importa `cp_model` somente quando o CP-SAT é executado. Se indisponível, fornece a indicação do arquivo de requisitos.

Retorno e efeitos. Módulo `cp_model` ou `ValueError`.

Referência: linha 20 do fonte identificado acima.

21.2 build_model

```
build_model(instance: Instance, choices: tuple[tuple[Choice, ...], ...], cost_cap: int |  
            None, makespan_cap: int | None, objective: str, *, include_energy: bool=True)
```

Responsabilidade. Construir a região viável CP-SAT e escolher o objetivo.

Funcionamento e contratos. Um intervalo opcional por job/máquina/modo, uma escolha por job, ausência de sobreposição e circuito por máquina. Adjacências ativam setup. `AddElement` representa energia por início e os tetos opcionais restringem energia e makespan. `include_energy=False` só é permitido com objetivo makespan e sem teto de custo: energia auxiliar é zero e deve ser reavaliada fora do modelo.

Retorno e efeitos. Modelo e variáveis de makespan, energia, presenças e inícios. O padrão mantém energia integral; a projeção da primeira fase conserva a região viável dos schedules.

Referência: linha 28 do fonte identificado acima.

21.3 decode_solution

```
decode_solution(instance: Instance, choices: tuple[tuple[Choice, ...], ...], solver,  
               presence: dict, starts: dict, makespan: int, energy: int | None)
```

Responsabilidade. Reconstruir escolhas e conferir a solução do solver.

Funcionamento e contratos. Cada job deve ter exatamente uma presença verdadeira. Resolve seu índice de escolha, verifica processamento/setups e makespan. Quando `energy=None`, o custo não foi modelado e será recalculado pelo chamador; com energia integral verifica igualdade.

Retorno e efeitos. Tupla de índices; `ValueError` se solver e avaliação não coincidem.

Referência: linha 132 do fonte identificado acima.

21.4 solve_front

```
solve_front(instance: Instance, *, output: Path, seconds_per_solve: float=60.0, workers:
    int=8, resume: bool=False, restart: bool=False, total_seconds: float=0.0, seed: int=0)
```

Responsabilidade. Enumerar os pontos Pareto com provas lexicográficas.

Funcionamento e contratos. Retoma somente arquivo com hash/escala compatíveis. Minimiza Cmax e depois TEC sob esse teto. Exige OPTIMAL em ambas as fases para arquivar. Reduz teto de energia em uma unidade e termina por INFEASIBLE ou custo negativo. Limites total/por chamada são cooperativos; workers e seed são registrados.

Retorno e efeitos. JSON com pontos provados, candidato, iterações, status, tempo e próximos limites. Fronteira completa exige a prova de que não há energia menor.

Referência: linha 154 do fonte identificado acima.

21.5 solve_front.save

```
solve_front.save(status: str, candidate: dict | None=None)
```

Responsabilidade. Salvar o estado de enumeração CP-SAT.

Funcionamento e contratos. Atualiza pares/índices provados, fronteira reconstruída, próximo teto, candidato e tempo acumulado. Marca pareto_proven somente com status complete.

Retorno e efeitos. Grava atomicamente relatório e devolve o dicionário.

Referência: linha 188 do fonte identificado acima.

21.6 solve_front.one_solve

```
solve_front.one_solve(cost_limit: int | None, makespan_limit: int | None, objective: str)
```

Responsabilidade. Executar uma otimização CP-SAT e avaliar seu resultado.

Funcionamento e contratos. Constrói modelo, limita a chamada pelo tempo restante, aplica workers/seed, registra status, bounds, branches e conflitos. Adia energia somente no primeiro makespan sem limite de custo. O custo real do candidato é sempre somado das escolhas decodificadas.

Retorno e efeitos. Status e candidato com energia real; FEASIBLE pode produzir candidato sem prova, UNKNOWN pode não fornecer candidato.

Referência: linha 199 do fonte identificado acima.

21.7 solver_reason

```
solver_reason(status, cp_model)
```

Responsabilidade. Traduzir um status não ótimo em motivo de parada.

Funcionamento e contratos. Distingue solução viável sem prova, desconhecido, modelo inválido e inviável. O motivo anterior é removido ao iniciar uma execução retomada.

Retorno e efeitos. Texto do motivo para o relatório.

Referência: linha 259 do fonte identificado acima.

21.8 main

```
main(argv: list[str] | None=None)
```

Responsabilidade. Executar CP-SAT pela linha de comando.

Funcionamento e contratos. Aceita arquivo/pasta, output-dir, limite por chamada, total, workers, seed e opções de retomada/reinício. Executa cada instância e imprime status e quantidade provada.

Retorno e efeitos. Código de saída 0 em sucesso; argparse relata parâmetros/arquivos inválidos.

Referência: linha 264 do fonte identificado acima.

22. Contagem do espaço de busca

src/analysis_metrics.py

Métricas combinatórias exatas e limites certificados, independentes do objetivo de energia.

Funções: 7. SHA-256: 7c6e46e87a63819fd736...

22.1 integer_summary

integer_summary(value)

Responsabilidade. Preservar e apresentar um inteiro muito grande.

Funcionamento e contratos. Converte o número integral para texto completo, conta dígitos e monta notação científica de seis algarismos para exibição. A notação curta não substitui o inteiro exato.

Retorno e efeitos. Dicionário exact, decimal_digits e scientific.

Referência: linha 22 do fonte identificado acima.

22.2 ordered_pairs

ordered_pairs(left_count, right_count, distance)

Responsabilidade. Contar pares de inícios que respeitam uma distância.

Funcionamento e contratos. Para x em $[0, A-1]$ e y em $[0, B-1]$, soma $y \geq x + \text{distance}$ por uma série aritmética. A fórmula evita percorrer H^2 pares.

Retorno e efeitos. Número inteiro exato, incluindo zero quando não há par admissível.

Referência: linha 29 do fonte identificado acima.

22.3 rule_bounds

rule_bounds(instance)

Responsabilidade. Calcular espaços irrestrito, one-hot, individual e relaxação por pares.

Funcionamento e contratos. $B = n * m * H * o$ define 2^B sem materializá-lo. $A = (m * H * o)^n$ e $D = \text{produto } D_j$ são inteiros completos. Pareia jobs disjuntos e exclui somente sobreposição dentro de cada par, permitindo toda interação restante e ignorando setups: $F \leq U \leq D$.

Retorno e efeitos. Dados por input, quantidades, reduções e definição do teto. H usa dias reais; percentuais têm denominadores explícitos.

Referência: linha 36 do fonte identificado acima.

22.4 count_feasible

```
count_feasible(instance, *, use_setups=True, deadline=None)
```

Responsabilidade. Contar todos os escalonamentos viáveis nas pequenas instâncias.

Funcionamento e contratos. Mantém contagem por máquina/subconjunto/último/término. Prefixos somam todos os termos compatíveis para cada modo, incorporando todas as esperas sem dominância. Libera estados já processados e combina contagens por partições. `use_setups=False` separa o efeito da ausência de sobreposição.

Retorno e efeitos. Contagem exata, prova, tempo e atualizações de slots. `Limite10jobs`; `TimeoutError` não publica total parcialmente calculado como exato.

Referência: linha 77 do fonte identificado acima.

22.5 count_feasible.partition

```
count_feasible.partition(count, subset)
```

Responsabilidade. Somar contagens de todas as partições de tarefas entre máquinas.

Funcionamento e contratos. Para uma máquina usa sua contagem de subset. Para mais máquinas enumera o subconjunto da última, multiplica sua contagem pela do complemento e soma. Cache evita recomputar o mesmo estado.

Retorno e efeitos. Inteiro de escalonamentos daquele estado.

Referência: linha 117 do fonte identificado acima.

22.6 backtracking_metrics

```
backtracking_metrics(report)
```

Responsabilidade. Distinguir cobertura de ramos e trabalho realmente executado.

Funcionamento e contratos. Valida `covered=folhas+propagação+setup+dominância` e `covered<=D`. Publica ramos por regras, por dominância, pendentes, decisões, folhas e pontos. Dominância pode excluir schedules viáveis sem interesse Pareto.

Retorno e efeitos. Dicionário de métricas; cobertura não equivale a percentual do Pareto nem total de schedules viáveis.

Referência: linha 132 do fonte identificado acima.

22.7 main

```
main()
```

Responsabilidade. Calcular e persistir métricas para todas as entradas.

Funcionamento e contratos. Desabilita a restrição local de dígitos inteiros para exportar produtos set2; salva bounds por input e conta exatamente apenas abaixo do limite. `--resume` reaproveita resultados por hash. A interrupção preserva métricas já concluídas.

Retorno e efeitos. `search_space.json` com 17 entradas; log do andamento e motivos para contagem não executada.

Referência: linha 152 do fonte identificado acima.

23. Validação, comparação e relatório

`src/analyze_results.py`

Gera snapshots, CSVs, gráficos e análise, com avaliação racional independente dos schedules publicados.

Funções: 9. SHA-256: f0a64c6ef9eb68997926...

23.1 hypervolume

`hypervolume(points, reference)`

Responsabilidade. Calcular área dominada 2D usando aritmética racional.

Funcionamento e contratos. Ordena pontos não dominados; soma retângulos entre energias consecutivas até a referência comum. Rejeita referência que não seja pior que os pontos. Linhas de gráficos não interpolam schedules existentes.

Retorno e efeitos. Fraction com HV exato; apresentação pode convertê-lo para float.

Referência: linha 23 do fonte identificado acima.

23.2 reference_point

`reference_point(instance)`

Responsabilidade. Construir referência HV fixa por instância.

Funcionamento e contratos. Usa $H+1$ e $1+\text{sum por job do maior custo possível de duração} \times \text{potência} \times \text{modo} \times \text{tarifa máxima}$. Esse teto domina qualquer schedule viável, inclusive pontos ainda não encontrados.

Retorno e efeitos. Par de makespan inteiro e energia racional.

Referência: linha 35 do fonte identificado acima.

23.3 validate_schedule

`validate_schedule(instance, point)`

Responsabilidade. Reavaliar independentemente um schedule salvo.

Funcionamento e contratos. Confere jobs exatos, índices, $\text{ceil}(\text{processing}/\text{speed})$, intervalo, horizonte e setups consecutivos. Energia é recomputada diretamente de tarifas e sobreposição de ponta em frações, sem usar `cost_units` gerado pelo solver.

Retorno e efeitos. True quando schedule está presente e correto; False para baseline sem schedule; ValueError para inconsistência.

Referência: linha 43 do fonte identificado acima.

23.4 triangular_violations

`triangular_violations(instance)`

Responsabilidade. Verificar a condição de equivalência com a separação entre pares do artigo.

Funcionamento e contratos. Para até 10 jobs testa todos os trios distintos por máquina e usa a menor duração do intermediário. Para entradas maiores declara não verificado, evitando trabalho cúbico injustificado.

Retorno e efeitos. Status, número de verificações e violações ou razão de não execução.

Referência: linha 70 do fonte identificado acima.

23.5 write_csv

`write_csv(path, rows)`

Responsabilidade. Exportar tabelas de análise com cabeçalho consistente.

Funcionamento e contratos. Usa `csv.DictWriter`, UTF-8 com BOM e `newline` correto para abertura em ferramentas locais. Inteiros de espaço são textos completos, não floats.

Retorno e efeitos. Grava CSV; tabela vazia não cria arquivo.

Referência: linha 86 do fonte identificado acima.

23.6 markdown_table

`markdown_table(headers, rows)`

Responsabilidade. Formatar registros paralelos como tabela Markdown.

Funcionamento e contratos. Monta cabeçalho, separador e linhas a partir dos valores recebidos. Os dados vêm do snapshot conferido.

Retorno e efeitos. Texto de tabela para o relatório editável.

Referência: linha 95 do fonte identificado acima.

23.7 freeze_reports

`freeze_reports(paths)`

Responsabilidade. Congelar os dados usados no relatório, tabelas e gráficos.

Funcionamento e contratos. Copia somente JSONs de resultados, sem checkpoints, preservando timestamps e origem. Comparações e figuras usam as mesmas cópias, evitando discrepância de cardinalidade enquanto solvers ativos gravam.

Retorno e efeitos. Caminhos congelados e mapeamento para fontes originais; snapshot constitui evidência do momento apresentado no PDF.

Referência: linha 99 do fonte identificado acima.

23.8 analyze

`analyze()`

Responsabilidade. Validar e consolidar os resultados existentes.

Funcionamento e contratos. Seleciona resultados por método/hash, inclui tentativas novas sem ponto e verifica coincidência de frentes completas, pertinência dos pontos provados e continuidade/energia da varredura. Decodifica candidatos CP-SAT e reavalia custos reais. CalculaHV, cobertura e convergência, grava hashes do código atual e chama gráficos/relatório.

Retorno e efeitos. `results_summary.json`, `search_space/results/convergence.csv`, gráficos principais e relatório Markdown. Valores sem prova permanecem candidatos; nenhuma biblioteca solver é executada.

Referência: linha 117 do fonte identificado acima.

23.9 build_report

`build_report(payload, space)`

Responsabilidade. Escrever a interpretação completa do snapshot.

Funcionamento e contratos. Descreve conformidade e diferenças do artigo, correção das três soluções, fórmulas/denominadores, contagem exata set1 e limites set2, resultados/tempos, podas, auditorias, protocolo e limites. Exclui alegações de benchmark ou contagem exata nas grandes entradas.

Retorno e efeitos. `docs/analise_metodos_exatos.md`, regenerável e editável após snapshot.

Referência: linha 227 do fonte identificado acima.

24. Controle das rodadas prolongadas

`src/run_experiments.py`

Inicia processos locais com logs, checkpoints preservados e manifestos; não fornece soluções de um método ao outro.

Funções: 4. SHA-256: 3f30e4bf91f3b755fefe...

24.1 `utc_now`

`utc_now()`

Responsabilidade. Registrar um horário inequívoco.

Funcionamento e contratos. Usa `datetime` com `timezone.utc`, sem depender do fuso da execução `host`.

Retorno e efeitos. `TextISO8601` em UTC.

Referência: linha 24 do fonte identificado acima.

24.2 `environment`

`environment()`

Responsabilidade. Registrar o contexto de execução.

Funcionamento e contratos. Obtém Python, executável, plataforma, CPUs lógicas e versões de OR-Tools, SciPy e Matplotlib. Dependência ausente vira `None`. Declara concorrência dos tempos.

Retorno e efeitos. Dicionário de proveniência; não altera ambiente.

Referência: linha 28 do fonte identificado acima.

24.3 `monitor`

`monitor(commands, manifest_path, *, parallel=False, previous=None)`

Responsabilidade. Executar uma fila e acompanhar resultados.

Funcionamento e contratos. Inicia subprocessos sem janela, separa logs, salva PIDs e comandos, lê outputs a cada 30s e guarda amostras de fronteira/estatísticas. Paralelismo é usado só nas duas buscas longas. `previous` preserva tentativas anteriores; falhas recebem `exit_code` e não viram sucesso.

Retorno e efeitos. Manifesto atualizado com histórico, horários/status; termina após todos os processos da fila.

Referência: linha 40 do fonte identificado acima.

24.4 main

`main()`

Responsabilidade. Preparar os comandos e orçamentos da rodada.

Funcionamento e contratos. Backtracking 6/7 parte de cópias dos próprios checkpoints. `--resume-run` mantém o ponto inicial e desconta tempo já executado do orçamento de 3 horas. Fronts roda DP 7..10 e CP 8..10 sequencialmente, com opções de limite/workers. Evita sobrescrever um manifesto sem opção explícita.

Retorno e efeitos. Cria/controla experimentos em `data/experiments/2026-09-30`. Não executar retomada enquanto controlador anterior está ativo; um escritor por output é requisito do protocolo.

Referência: linha 84 do fonte identificado acima.

25. Testes dos contadores combinatórios

tests/test_analysis_metrics.py

Confere as contagens de regras e relaxações com enumeração completa de casos pequenos.

Funções: 2. SHA-256: cd2a3797ff05e8c372ea...

25.1 CountingTest.test_exact_counts_against_enumeration

CountingTest.test_exact_counts_against_enumeration(self)

Responsabilidade. Conferir F e a ausência de sobreposição contra enumeração.

Funcionamento e contratos. Enumera todas as escolhas das seis fixtures, conta ausência de overlap e setups válidos, e compara com as duas DPs. Verifica $F \leq U \leq D$.

Retorno e efeitos. Teste passa somente com identidade dos inteiros em todas as fixtures.

Referência: linha 13 do fonte identificado acima.

25.2 CountingTest.test_ordered_pairs

CountingTest.test_ordered_pairs(self)

Responsabilidade. Conferir a fórmula analítica dos inícios.

Funcionamento e contratos. Enumera tamanhos e gaps de 0a5 e conta diretamente $y \geq x + \text{gap}$, incluindo domínios vazios.

Retorno e efeitos. Teste da identidade exata para todos os pequenos pares.

Referência: linha 30 do fonte identificado acima.

26. Teste de persistência no Windows

tests/test_atomic_output.py

Verifica recuperação de bloqueio transitório sem perder a última versão de saída.

Funções: 2. SHA-256: 9718f07ae4d4dc335b82...

26.1 AtomicOutputTest.test_transient_reader_lock_and_permanent_denial

AtomicOutputTest.test_transient_reader_lock_and_permanent_denial(self)

Responsabilidade. Conferir recuperação e propagação da falha permanente.

Funcionamento e contratos. Grava versão1, simula duas negações seguidas de troca real, verifica versão2. Depois simula negação permanente e confirma que versão2 continua intacta. Executado somente noWindows.

Retorno e efeitos. Teste com temporários isolados e nenhum arquivo de experimento alterado.

Referência: linha 15 do fonte identificado acima.

26.2

AtomicOutputTest.test_transient_reader_lock_and_permanent_denial.locked_then_replace

AtomicOutputTest.test_transient_reader_lock_and_permanent_denial.locked_then_replace(source
, target)

Responsabilidade. Simular bloqueio transitório do destino.

Funcionamento e contratos. Conta tentativas, lança PermissionError nas duas primeiras e delega ao os.replace real depois.

Retorno e efeitos. Efetua a troca somente na terceira chamada para exercitar a recuperação.

Referência: linha 22 do fonte identificado acima.

27. Testes da certificação CP-SAT

tests/test_cpsat.py

Compara com oráculo, valida projeção sem energia, custo real de candidatos e retomada dos motivos de parada.

Funções: 7. SHA-256: 718249ea4c8f7720b766...

27.1 CpSatTest.setUpClass

CpSatTest.setUpClass(cls)

Responsabilidade. Verificar disponibilidade do OR-Tools.

Funcionamento e contratos. Tenta carregar cp_model uma vez antes da classe. Se ausente, marca os testes dessa biblioteca como opcionais e ignorados.

Retorno e efeitos. Prepara a classe ou unittest.SkipTest.

Referência: linha 20 do fonte identificado acima.

27.2 CpSatTest.test_same_complete_front_as_oracle

CpSatTest.test_same_complete_front_as_oracle(self)

Responsabilidade. Comparar toda fronteira CP-SAT com a enumeração.

Funcionamento e contratos. Resolve seis fixtures, exige complete e pareto_proven e compara makespan/custo inteiro exatamente ao oráculo.

Retorno e efeitos. Prova experimental de correspondência em casos pequenos.

Referência: linha 26 do fonte identificado acima.

27.3 CpSatTest.test_resume_between_proven_points

CpSatTest.test_resume_between_proven_points(self)

Responsabilidade. Verificar retomada da enumeração Pareto.

Funcionamento e contratos. Obtém frente completa, salva somente o primeiro ponto e próximo teto, retoma e compara com a frente original.

Retorno e efeitos. Teste sem duplicar pontos ou perder melhora estrita.

Referência: linha 40 do fonte identificado acima.

27.4 CpSatTest.test_makespan_projection_and_energy_guard

CpSatTest.test_makespan_projection_and_energy_guard(self)

Responsabilidade. Verificar equivalência de otimização com e sem tabelas de energia.

Funcionamento e contratos. Nas seis fixtures, minimiza Cmax nos dois modelos e compara com oráculo. Decodifica custo real e rejeita omissão de energia com limite de custo ou objetivo energético.

Retorno e efeitos. Teste da projeção segura e das guardas.

Referência: linha 57 do fonte identificado acima.

27.5 CpSatTest.test_feasible_candidate_has_actual_energy

`CpSatTest.test_feasible_candidate_has_actual_energy(self)`

Responsabilidade. Verificar candidato FEASIBLE com energia real.

Funcionamento e contratos. Simula statusFEASIBLE após obter valores reais de solver; confirma custo positivo somado das escolhas e ausência de pontos provados. A energia auxiliar zero não pode aparecer comoTEC real.

Retorno e efeitos. Teste da distinção candidato/prova.

Referência: linha 76 do fonte identificado acima.

27.6 CpSatTest.test_feasible_candidate_has_actual_energy.feasible_status

`CpSatTest.test_feasible_candidate_has_actual_energy.feasible_status(solver, model)`

Responsabilidade. Simular solução sem certificado ótimo.

Funcionamento e contratos. ChamaSolve original para preencher os valores e retornaFEASIBLE ao wrapper. Isso permite verificar o fluxo incompleto deterministicamente.

Retorno e efeitos. StatusFEASIBLE com um schedule real.

Referência: linha 80 do fonte identificado acima.

27.7 CpSatTest.test_total_deadline_and_fresh_resume_reason

`CpSatTest.test_total_deadline_and_fresh_resume_reason(self)`

Responsabilidade. Verificar orçamento total e razão de parada nova.

Funcionamento e contratos. Usa orçamento minúsculo para interromper antes da primeira prova, retoma com10s e exige frente completa com no_cheaper_solution no lugar do motivo anterior.

Retorno e efeitos. Teste de persistência dos pontos e atualização de metadados.

Referência: linha 96 do fonte identificado acima.